COMSATS University Islamabad, Sahiwal Campus

FinEase

(AI-Powered Financial Inclusion System)

A project presented to

COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Abdul Ahad	**CIIT/FA22-BCS-002/SWL**
Abdullah Khaleeq	**CIIT/FA22-BCS-005/SWL**
Rehan Ali Ch	**CIIT/FA22-BCS-104/SWL**

DECLARATION

We hereby declare that this project, neither as a whole nor in part, has been copied from any source. It is further declared that we have developed this software and the accompanying report entirely on the basis of our personal efforts under the supervision of **Ms. Raheela Shahzadi** and co supervision of **Mr. Ahmad Farid** at COMSATS University Islamabad, Sahiwal Campus. If any part of this project is proved to be copied from any source or found to be a reproduction of some other work, we will stand by the consequences. No portion of the work presented herein has been submitted in support of any application for any other degree or qualification at this or any other university or institute of learning.

Abdul Ahad	**Abdullah Khaleeq**	**Rehan Ali Ch**
----------	----------	----------

CERTIFICATE OF APPROVAL

It is hereby certified that the final year project of BS (CS) titled "**FinEase** (Financial Inclusion Mobile Application)" was developed by **Abdul Ahad** (CIIT/FA22 BCS 002/SWL), **Abdullah Khaleeq** (CIIT/FA22 BCS 005/SWL), and **Rehan Ali Ch** (CIIT/FA22 BCS 104/SWL) under the supervision of **Ms. Raheela Shahzadi** and co supervision of **Mr. Ahmad Farid**, and that in their opinion it is fully adequate in scope and quality for the degree of Bachelor of Science in Computer Science.

Supervisor

Ms. Raheela Shahzadi

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

FinEase is a mobile first financial management and financial inclusion application designed and developed for common Pakistanis. The application tackles an important problem: despite growing smartphone penetration, most of many Pakistanis cannot access money management help, savings tools, welfare program information, or affordable financial advice. FinEase solves this by providing AI powered financial platform that runs on Android 8.0+ devices with as little as 2 GB RAM.

FinEase is a mobile-first financial inclusion application designed for uneducated and low-income communities in Pakistan. The app was built because while smartphone penetration is growing, only 26% possess basic financial literacy. To help with this, FinEase delivers a platform that is accessible even on low-end Android devices with as little as 2 GB of RAM.

The application acts as a personal finance helper, offering eleven core modules including a financial literacy hub, savings goal trackers, an AI-powered budget advisor and a directory for NGO and welfare program. It also integrates gamification and community forums to keep users engaged over time. A central feature is the AI chatbot, powered by Google's Gemini API, which provides real-time conversational guidance to users who might not be able to afford professional financial advice.

Technically, FinEase uses a Flutter frontend and a Firebase cloud backend for authentication, real-time data management, and serverless functions. Built using an Agile approach, the system uses a multi-layer client-cloud design. The reliability of the platform is backed by thorough testing, with all 44 automated test cases achieving a 100% pass rate, making it a reliable tool that helps people manage their money.

Acknowledgement

All praise is due to Almighty Allah, the Most Gracious and the Most Merciful, who bestowed upon us the knowledge, ability, and perseverance to accomplish this challenging yet rewarding project. Every step of this journey from the initial concept through design, implementation, testing, and documentation was made possible by His guidance and blessings.

We owe our deepest gratitude to our project supervisor, **Ms. Raheela Shahzadi**, Lecturer in the Department of Computer Science at COMSATS University Islamabad, Sahiwal Campus. Her expertise, patience, consistent guidance, and timely feedback were invaluable at every stage from scope definition through final implementation and documentation. Without her dedicated supervision, FinEase would not have been possible.

We are equally grateful to our co supervisor, **Mr. Ahmad Farid**, for his insightful technical advice, particularly regarding the system architecture, Firebase integration, and AI module design. His suggestions on the multi tier architecture, Firestore data model, and Gemini API integration helped us make better informed decisions throughout development.

We would like to extend our sincere thanks to the faculty of the **Department of Computer Science at COMSATS University Islamabad, Sahiwal Campus**, whose teaching over the past four years equipped us with the knowledge and skills needed to undertake a project of this scope. Special thanks are due to the **Final Year Project Committee** for their constructive evaluation feedback at each milestone review.

Finally, we are deeply indebted to our families for their unwavering moral and emotional support throughout our academic journey. Their sacrifices, encouragement, prayers, and belief in our abilities have been the foundation upon which this achievement rests.

Abdul Ahad **Abdullah Khaleeq** **Rehan Ali Ch**

---------- ---------- ----------

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
BISP	Benazir Income Support Program
CRUD	Create, Read, Update, Delete
FCM	Firebase Cloud Messaging
FR	Functional Requirement
LLM	Large Language Model
MFI	Microfinance Institution
NFR	Non Functional Requirement
NGO	Non Governmental Organization
NoSQL	Non Relational Database
OOD	Object Oriented Design
OOP	Object Oriented Programming
OTP	One Time Password
PKR	Pakistani Rupee

Abbreviation	Full Form
RBAC	Role Based Access Control
RTL	Right to Left (text direction)
SDD	Software Design Description
SDK	Software Development Kit
SRS	Software Requirements Specification
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
FCF	Firebase Cloud Functions
KPI	Key Performance Indicator

Table of Contents

Chapter 1 Introduction

List of Figures

List of Tables

Chapter 1

Introduction

1 Introduction

FinEase is a modern mobile first financial inclusion application designed to address the compound financial exclusion crisis faced by underserved communities in Pakistan. The system integrates financial literacy education, savings tracking, loan simulation, government welfare program discovery, AI powered budgeting, and a Gemini powered bilingual chatbot into a single, accessible Android application that runs on any Android 8.0+ smartphone with as little as 2 GB RAM. This report documents the complete development lifecycle of FinEase, including its academic relevance, problem context, literature foundations, requirements, design, implementation, and testing.

Pakistan's financial landscape is characterized by a stark digital divide. Smartphone penetration in urban centers exceeds 65% yet the depth of financial services usage remains alarmingly shallow. The majority of mobile phone owners in underserved areas use their devices for social media and entertainment not for savings, budgeting, or financial planning. The OECD/INFE 2021 Financial Literacy Survey found that only 26% of Pakistani adults can correctly answer basic questions about compound interest and inflation. FinEase was conceived to close this gap by transforming the smartphone into a comprehensive personal finance companion that is accessible, bilingual, offline capable, and powered by state of the art AI.

1.1 Brief

FinEase short for Financial Ease brings together financial literacy education, practical savings tracking, loan simulation, government welfare program discovery, AI powered budgeting guidance, peer community support, and a gamified reward system into a single, accessible Android application. The platform is developed using Flutter (Google's cross platform framework using the Dart language) and Firebase (Google's cloud platform). The AI Chatbot module leverages Google's Gemini large language model through Firebase Cloud Functions, enabling bilingual (Urdu and English) conversational financial assistance. The application is designed specifically for users in Pakistan, focusing on simplicity, affordability, and accessibility for individuals with limited financial knowledge. Its scalable and user-friendly architecture ensures reliable performance while supporting future growth and feature expansion.

1.2 Relevance to Course Modules

The development of FinEase draws meaningfully upon knowledge acquired across multiple courses in the BS Computer Science curriculum at COMSATS University Islamabad, Sahiwal Campus.

1.2.1 Software Engineering Methodologies

FinEase was built using a range of software engineering methods and techniques. An Agile-inspired incremental model was used to manage development, allowing the team to deliver and test working features at each stage. Requirements were gathered and documented using a Software Requirements Specification (SRS), and the system design was recorded in a Software Design Description (SDD). UML diagrams including use case, class, sequence, and activity diagrams were used to plan and communicate the system structure. Functional and non-functional requirements were formally defined and tracked throughout the project. A V-model testing approach was applied, with unit, integration, functional, and system testing carried out at each development phase. A Requirements Traceability Matrix was maintained to ensure every requirement was linked to a design component and a test case.

1.2.2 Mobile Application Development

The Flutter framework's widget based architecture, Provider state management, named route navigation, inline form validation, responsive layouts using MediaQuery, and Android specific configuration (AndroidManifest.xml, google-services.json, ProGuard) all reflect coursework in this area. The Material Design 3 component library, bilingual support with RTL text rendering for Urdu, and the step by step UX design are direct applications of mobile specific internationalization and accessibility techniques covered in this course.

1.2.3 Database Systems

The Data Dictionary reflects systematic database design mapping each domain entity to a Firestore collection with clearly defined fields, data types, cardinality, and relationships. The decision to use Firestore's denormalized document model for certain entities embedding quiz questions within quiz documents, embedding expense breakdowns within budget plan documents directly applies normalization trade off analysis. Firebase Security Rules implement row level access control policies analogous to those taught in the Database Systems course.

1.2.4 Artificial Intelligence

The AI Budget Advisor implements a rule based expert system using the 50/30/20 personal finance rule. The AI Chatbot module integrates Google's Gemini large language model via Firebase Cloud Functions, demonstrating applied knowledge of natural language processing, prompt engineering, and LLM utilization for domain specific applications. The system prompt engineering that constrains Gemini to act as a FinEase specific Pakistani personal finance advisor generates responses in the user's preferred language reflects the understanding of LLM capabilities taught in the course.

1.2.5 Human Computer Interaction

FinEase's target users financially underserved individuals with potentially limited digital and formal literacy require an interface that minimizes cognitive load, provides clear feedback, uses familiar metaphors, avoids technical jargon, and accommodates users who may never have used a financial application before. HCI principles evident include minimizing memory load (consistent navigation patterns), visible system status (progress bars, loading indicators), error prevention (inline form validation, confirmation dialogs before irreversible actions), and designing for diverse users (bilingual support, large tap targets, minimum font sizes).

1.2.6 Object Oriented Programming

FinEase is entirely implemented using OOP principles in Dart. The domain model (User, SavingsGoal, Lesson, Quiz, NGOProgram, LoanSimulation, BudgetPlan, Reward, ChatbotSession), service classes, and repository classes are all structured as objects with encapsulated state and behavior. Inheritance is demonstrated in the User and Admin relationship. Abstract classes define repository contracts. Polymorphism is evident in how different module screens all conform to Flutter’s Widget interface.

1.2.7 Data Structures and Algorithms

Data Structures and Algorithms formed a foundational layer of FinEase, contributing to the correctness, efficiency, and reliability of multiple modules throughout the system. Although FinEase is not a compute intensive algorithmic application, the discipline underpins nearly every significant operation in the platform. The loan amortization algorithm applies mathematical recurrences to compute monthly installments in O(1) time, while the savings goal progress update

relies on atomic transactional logic to prevent race conditions and maintain data integrity. The budget advice generation algorithm uses sorted ranking of expense deviations an O(n log n) operation to prioritize the most impactful advice items among up to five categories. Firestore's document oriented storage model maps directly to associative map structures taught in the Data Structures course: each Firestore document is effectively a key value map, and the fourteen top level collections form an indexed dataset that supports efficient lookup, filtering, and pagination. The chatbot conversation history is stored as a bounded FIFO array of message pairs a queue based structure naturally limiting context to ten message pairs. The rewards system daily cap logic depends on comparison and arithmetic operations characteristic of algorithmic reasoning. Overall, Data Structures and Algorithms provided the conceptual toolkit for making sound implementation decisions at every point in the system where performance, correctness, and data organization intersect.

1.2.8 Computer Networks

Computer Networks is relevant to FinEase because the app depends on communication between the client and cloud services. The Android app connects to Firebase services using HTTPS, which requires basic understanding of how client–server communication works and how data is sent securely using TLS. The AI Chatbot and Budget Advisor use a Cloud Function, which works like sending a request and receiving a response over the network.

The app also handles weak or no internet by using offline support. Firestore stores data locally so the app can still work when the network is unavailable. The Gemini API key is kept in Cloud Functions instead of the app to prevent it from being exposed over the network.

1.2.9 Software Quality Assurance

Software Quality Assurance (SQA) influenced how testing and quality were handled in FinEase. A requirements traceability matrix was used to make sure every requirement is linked to design and test cases, so nothing is missed. The project includes a full test suite with unit tests, widget tests, integration tests, and Firebase Security Rules tests, all used to check the system properly.

Different types of testing were applied: unit testing for small parts of the code, functional testing for requirements, and integration testing for how different parts work together. Firestore transactions were used to avoid data errors by making operations like deposits atomic. Security

Rules were also tested to ensure correct access control instead of just assuming they work. Overall, SQA helped ensure quality was considered throughout development, not just at the end.

1.3 Project Background

Pakistan's financial inclusion landscape presents a complex picture of rapid technological growth alongside persistent economic inequality. The World Bank's 2021 Global Findex Database reports that only 21% of Pakistani adults maintain a formal bank account one of the lowest rates in South Asia. While mobile money adoption has grown through platforms like JazzCash and Easypaisa (collectively over 100 million registered accounts), the depth of financial inclusion remains shallow: most users limit activity to simple transfers and bill payments rather than savings, credit, or investment products.

Financial literacy, the knowledge and skills needed to make informed financial decisions is a prerequisite for meaningful financial inclusion. Yet surveys consistently show very low financial literacy rates among Pakistani adults, particularly women, rural populations, and those with limited formal education. The COVID 19 pandemic (2020–2022) accelerated smartphone adoption but the corresponding growth in digital financial literacy lagged significantly. It is against this backdrop that FinEase was conceived. The project team observed firsthand the financial challenges faced by their communities: family members unaware of NGO support programs, relatives falling prey to exploitative informal lenders, neighbours unable to save consistently due to lack of tools and knowledge.

1.4 Literature Review

1.4.1 Financial Inclusion: Theory and Global Evidence

Financial inclusion means giving people access to useful and affordable financial services. It is important for economic growth and reducing poverty. According to the World Bank, about 1.7 billion adults worldwide do not have bank accounts, and mobile phones can help improve access to financial services.

Research shows that financial inclusion helps people manage their money better, save, and handle risks. In Pakistan, one major issue is lack of financial knowledge and information, especially for

small borrowers. FinEase addresses this problem through its Financial Literacy Hub, which helps users better understand financial concepts and services.

1.4.2 Mobile Financial Services in Developing Countries

Studies on Kenya's M-Pesa show that mobile money can improve saving habits, reduce risk, and help lower poverty. In Pakistan, the main barriers to mobile banking are security concerns, low digital literacy, and language issues. FinEase addresses these problems by using secure communication and access rules for safety, a simple step-by-step interface for ease of use, and support for both Urdu and English to overcome language barriers.

1.4.3 Financial Literacy Interventions

Research shows that people with better financial literacy tend to save more and build more wealth. It also shows that learning at the moment of need (just-in-time learning) is more effective than general classroom teaching. FinEase uses this idea by combining learning with practical tools like savings goals and loan simulations, so users can apply what they learn immediately.

Studies also show that simple, action-focused content works best when combined with reminders and motivation. FinEase includes features like savings reminders, basic gamification, and social elements to encourage better financial habits.

1.4.4 Gamification and AI in Financial Applications

Research shows that gamified financial education helps people remember 20–30% more compared to traditional methods. This supports FinEase's rewards and gamification system, including daily point limits to keep users engaged in a meaningful way. Studies also show that chatbot-based financial tools are more effective than static FAQs, especially for users with less knowledge. This supports the use of the Gemini-powered chatbot in FinEase. The Budget Advisor uses the simple 50/30/20 rule because research shows that easy and memorable rules work better than complex methods, especially for people with low financial literacy.

1.5 Analysis from Literature Review

The literature review highlights five key ideas that shaped FinEase's design. First, learning works best when combined with action, so FinEase includes tools like savings goals, loan simulation, and budgeting alongside lessons.

Second, language is a major barrier, so the app supports both Urdu and English. Third, behaviour change is more important than just giving information, so features like reminders, streaks, and badges are included to motivate users. Fourth, chatbots are more helpful for users with less knowledge, which supports the use of the Gemini-powered chatbot. Fifth, simple rules work better than complex methods, so the Budget Advisor uses the easy 50/30/20 rule.

1.6 Methodology and Software Lifecycle

1.6.1 Object Oriented Design Methodology

FinEase employs Object Oriented Design (OOD) as its core software design methodology. OOD was chosen because the real world domain maps naturally to objects: a User has Profile, creates SavingsGoals, takes Lessons, attempts Quizzes, runs LoanSimulations, generates BudgetPlans, earns Rewards, and engages with ChatbotSessions. The three layer object model domain objects (data), service objects (business logic), repository objects (data access) ensures any component can be independently tested, modified, or replaced.

1.6.2 Agile Inspired Incremental Development Model

The development lifecycle follows an Agile inspired incremental model, as shown in Figure 1.1, with four increments aligned to semester milestones. This model was chosen over waterfall because requirements evolved progressively through supervisor evaluations; AI modules benefited from early prototyping and iterative refinement; and the semester schedule naturally maps to incremental delivery.

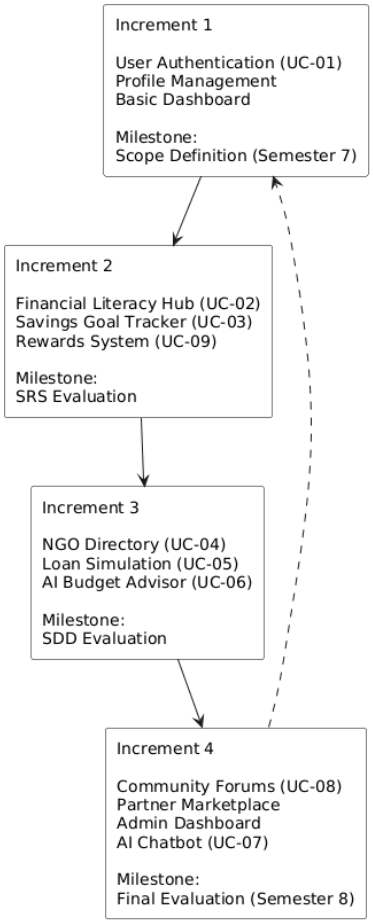

Figure 1.1: Agile Incremental Development Model used for FinEase

Figure 1.1 illustrates an Agile Incremental Development Model where the system is developed in four structured phases, each adding new functionality while building on the previous work. In the first increment, core features such as user authentication, profile management, and a basic dashboard are implemented, establishing the project scope. The second increment introduces additional modules like the financial literacy hub, savings tracker, and rewards system, followed by SRS evaluation. In the third increment, advanced features such as the NGO/Welfare directory, loan simulation, and AI budget advisor are developed and evaluated through SDD. Finally, the fourth increment completes the system with community forums, partner marketplace, admin dashboard, and AI chatbot, leading to the final evaluation. This model ensures continuous development, testing, and improvement until the system is fully completed.

1.6.3 Rationale for Technology Choices

Flutter was selected because a single Dart codebase targets Android and iOS; Flutter's rich Material Design 3 widget library suits the Android ecosystem; hot reload accelerates iterative UI development; and comprehensive documentation supported a student team. Firebase was selected for its serverless infrastructure, generous free tier, Firestore offline persistence, native Cloud Functions integration, and seamless connection to the Gemini API through Google's unified ecosystem. The Gemini API was chosen over alternatives for cost efficiency, strong Urdu language performance, and Firebase integration simplicity.

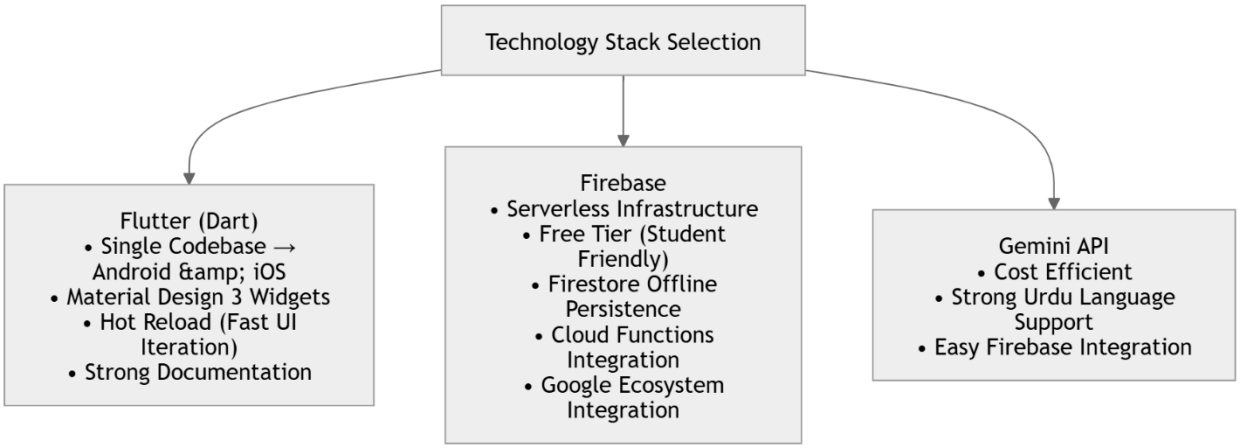

Figure 1.2: FinEase Technology Stack Overview

Figure 1.2 presents the full FinEase technology stack, showing the Flutter/Dart client layer, Firebase services (Authentication, Firestore, Cloud Functions, FCM), and the Gemini AI API, along with the communication protocols connecting each tier.

Chapter 2

Problem Definition

2 Problem Definition

This chapter provides a precise and evidence based definition of the problem FinEase addresses. It articulates the core problem statement, identifies specific challenges that motivated the project's feature set, enumerates deliverables and development requirements, and provides a critical assessment of the current system landscape in Pakistan's financial inclusion space. The problem definition serves as the foundation from which all requirements, design decisions, and implementation choices are derived.

2.1 Problem Statement

Pakistan faces a compound financial inclusion crisis characterized by low financial literacy, limited access to formal financial tools, inadequate awareness of government welfare programs, and exploitative informal lending, all disproportionately affecting low income and semi urban populations who increasingly own smartphones but lack the financial knowledge and tools to use them productively. Despite over 65% smartphone penetration in urban areas, the digital financial services landscape fails this demographic in four critical ways: existing platforms are transactional rather than educational; government welfare portals are inaccessible to low literacy users; no affordable personalized financial advisory service exists for low income households; and informal savings mechanisms offer no tracking, analysis, or accountability features.

2.1.1 Specific Challenges Addressed

Financial Illiteracy: According to the OECD/INFE 2021 Financial Literacy Survey, only 26% of Pakistani adults can correctly answer basic questions about compound interest, inflation, and risk diversification. This gap leads to poor decisions including over borrowing, under saving, and susceptibility to financial fraud. The Financial Literacy Hub directly addresses this through structured lessons, interactive quizzes, and an evidence based gamification system.

Limited Awareness of Welfare Programs: Programs such as BISP, Ehsaas, Zakat, and numerous NGOs collectively allocate hundreds of billions of rupees annually for poverty alleviation, yet a significant portion of eligible beneficiaries are unaware or unable to navigate the application process. FinEase addresses this through the centralized, searchable NGO Directory with eligibility criteria, step by step guidance, and real time status tracking.

Exploitative Informal Lending: The absence of accessible formal loan education tools leaves low income borrowers vulnerable to informal lenders charging 30–50% annual interest rates. FinEase's Loan Simulation Tool enables users to understand formal loan terms and make informed borrowing decisions before approaching a financial institution.

Language Accessibility Barrier: Virtually all financial applications in Pakistan use English as their primary interface language, structurally excluding most of the population for whom Urdu is the primary literacy language. FinEase's full Urdu/English bilingual interface with proper Noto Nastaliq Urdu font rendering and RTL text direction removes this barrier entirely.

2.2 Deliverables and Development Requirements

2.2.1 Project Deliverables

FinEase Android Application: A fully functional Flutter based Android application implementing all eleven modules, deployable as a release APK. Software Requirements Specification (SRS): A professionally aligned document capturing all functional requirements, non functional requirements, use cases, and constraints. Software Design Description (SDD): A professionally aligned design document covering system architecture, UML models, data dictionary, algorithms, and human interface design. Final Year Project Report: This document covering the complete project lifecycle. GitHub Repository: A version controlled codebase with a complete commit history. Project Presentation: A 20 slide power point presentation that will give demonstration of the working application.

The FinEase project encompasses a comprehensive suite of development deliverables that collectively constitute the complete final year project submission. Each deliverable corresponds to a distinct phase of the software development lifecycle and addresses one or more stakeholders' requirements. The primary software deliverable is the FinEase Android Application, a fully functional, Flutter based mobile application implementing all eleven core modules, deployable as a signed release APK compatible with Android 8.0 and higher devices with a minimum of 2 GB RAM. Alongside the software product, the project produces a Software Requirements Specification (SRS), capturing all functional requirements, non functional requirements, use cases, constraints, and the complete requirements traceability matrix. The Software Design Description (SDD), documents the system architecture, all UML design models, the Firestore data dictionary,

algorithm specifications, and the human interface design specification. This Final Year Project Report constitutes the comprehensive narrative document covering the complete project lifecycle from literature review through testing and future work. Additional deliverables include a fully version controlled GitHub repository with a complete commit history reflecting all four increments, and a structured project presentation with working application demonstration. Together, these deliverables demonstrate the complete application of software engineering practice in producing a professionally documented, tested, and deployable software product.

2.2.2 User Interface Design

The user interface of FinEase is designed to serve financially underserved users who may have limited digital literacy and no prior experience with financial applications. The design adheres to Material Design 3 guidelines and prioritizes accessibility, clarity, and ease of navigation throughout all eleven modules. The primary color palette uses Teal (#009688) and Green (#4CAF50) to convey trust and financial positivity, with a minimum body font size of 14sp and heading sizes of 18sp or higher to ensure readability. Contrast ratios maintain a at least 4.5:1 across all text and background combinations. All interactive tap targets are sized at a minimum of 48x48 dp to accommodate users with varying levels of fine motor precision. Every primary user task from creating a savings goal to querying the AI chatbot is reachable within one tap from the main dashboard. The bilingual interface supports full Urdu (Noto Nastaliq Urdu font, RTL text direction) and English (Nunito font, LTR text direction) based on the user's language preference. Inline form validation provides immediate feedback within 500 milliseconds of invalid input, reducing user frustration and guiding correct data entry. Consistent navigation patterns, loading indicators and confirmation dialogs minimize cognitive load and build user confidence across the application.

2.2.3 User Authentication and Registration

FinEase implements a secure and inclusive user authentication and registration mechanism that accommodates the diverse connectivity and literacy levels of its target population. New users register using an email address. The authentication flow is managed entirely by Firebase Authentication, which handles verification with built in security guarantees including HTTPS encrypted transmission. On first registration, a Firestore user document is created with the user's name and language preference. Role-based access control is enforced via Firebase Custom Claims,

ensuring that admin users access the admin dashboard while regular users are restricted to their personal data. Firebase persistent authentication tokens maintain sessions across application restarts, eliminating the need for repeated logins.

2.2.4 Admin Panel

The FinEase Admin Panel provides a dedicated interface for administrative users to manage and govern the platform's content and user interactions. Admin users are distinguished from regular users through Firebase Custom Claims assigned via the Firebase Admin SDK, and all admin only Firestore paths are protected by Security Rules that reject access from user role accounts. Through the Admin Panel, administrators can add, edit, approve, and remove NGO program listings, ensuring that only verified welfare programs are visible to end users in the NGO Directory. The Admin Panel also supports moderation of community forum threads and user management functions. Analytics capabilities within the admin interface provide visibility into platform activity including lesson completion rates, savings goal creation trends and chatbot usage, enabling data driven decisions about content quality and program relevance.

2.2.5 AI Chatbot Module

The FinEase AI Chatbot is a conversational financial guidance assistant powered by Google's Gemini large language model, delivered through a Firebase HTTPS Callable Cloud Function to ensure API key security. The chatbot accepts queries in both Urdu and English and generates responses in the user's preferred language, making conversational AI accessible to users who are not comfortable in English. The chatbot can answer questions about FinEase features, explaining financial concepts, describing NGO program eligibility criteria, and providing loan and savings guidance. A Gemini system prompt constrains the model to act as a Pakistani personal finance advisor focused on FinEase topics, preventing off topic responses. Conversation context is maintained through a rolling window of the last ten message pairs stored in a Firestore ChatbotSession document, enabling contextually aware responses across multiturn conversations. A typing indicator and quick action suggestion chips improve the first interaction experience for new users. Graceful error handling ensures that network failures produce helpful fallback messages rather than application crashes. The chatbot must respond within eight seconds on a 4G connection.

2.2.6 AI Financial Coach

The AI Financial Coach is a key feature of FinEase that turns it from a simple expense tracker into a smart financial advisor. Instead of just showing spending data, it analyzes the user's finances and provides useful suggestions in real time. It uses the Gemini API to generate personalized advice based on the user's spending, budget, and expenses. The system prepares a summary of the user's financial data and sends it to the AI, which then returns simple and relevant advice in PKR.

The module works in two ways. First, it has rule-based alerts that instantly notify users when certain conditions are met, such as overspending on food, exceeding a budget, or saving too little. Second, it includes a chatbot where users can ask questions like "How can I save more?" and get helpful responses. The interface includes a chat screen with suggested questions, a loading indicator while waiting for responses, and a Coach Advice Card on the home screen that shows quick tips. Overall, this feature makes FinEase feel like a personal financial advisor rather than just a tracking app.

2.2.7 Financial Literacy Module

The Financial Literacy Module is FinEase's core educational component, addressing the root cause of financial exclusion by building knowledge and practical decision making skills in users. The module provides structured financial literacy lessons organized by topic category and difficulty level (Beginner, Intermediate, Advanced), rendered from Markdown formatted Firestore content to support rich text including lists, bold text, and embedded references. All lessons are available in both Urdu and English, fetched with a Firestore query filtered by the user's language preference. Each lesson is followed by a quiz of at least five questions, evaluated client side and scored as a percentage. Users scoring 80% or above receive reward points through an atomic Firestore transaction that enforces the daily points cap. Lesson progress is tracked per user (unlocked, in progress, completed) and previously accessed lessons remain available offline through Firestore's persistent cache. The gamification integration of earning points and badges for lesson completion transforms passive reading into active, reward reinforced learning.

2.2.8 Savings Goal Tracker Module

The Savings Goal Tracker addresses the absence of structured savings tools for low income users by providing a simple, motivated, and analytically transparent goal management system. Users can

create savings goals by specifying a title, target amount in Pakistani Rupees, and target date, establishing a clear financial objective with a defined timeline. Deposit recording uses Firestore atomic transactions to prevent race conditions and ensure that progress calculations are always accurate. Over deposits are automatically capped at the target amount, and the goal status transitions automatically to Completed when the target is reached, triggering a reward and a completion animation to reinforce the achievement. Users can cancel goals, with cancelled goals preserved in a historical view. Input validation prevents goals with past dates or zero targets. The real time Firestore stream subscription ensures that progress bars and status indicators update reactively without requiring explicit refresh. The algorithm uses Firestore atomic transactions to ensure accuracy.

2.2.9 NGO and Welfare Directory Module

The NGO and Welfare Directory addresses the critical problem of low awareness of government and NGO welfare programs among eligible beneficiaries by providing a centralized, searchable, and application enabled directory of support programs. The directory maintains program records with names, categories, eligibility criteria, and step by step application guidance, managed by administrators through the Admin Panel. Users can filter programs by category (Financial Aid, Health, Education, Housing, Food Security) and search by program name to quickly locate relevant support. The guided application submission flow breaks the application process into clear steps, reducing the literacy and process barriers that prevent eligible users from applying.

2.2.10 Loan Simulation Module

The Loan Simulation Module addresses exploitative informal lending by equipping users with the knowledge to evaluate formal loan terms before committing to a borrowing decision. The module accepts three inputs principal amount, annual interest rate, and repayment tenure in months and applies the standard amortization formula to compute the monthly installment, total repayable amount, and total interest cost, all formatted in Pakistani Rupees. The zero interest rate edge case is handled separately using the simplified formula $M = P / n$. An affordability indicator compares the computed monthly installment against the user's stored income estimate and displays a green (affordable), amber (moderate), or red (potentially straining) indicator to contextualize affordability. Since the computation is entirely client side, the module functions without internet connectivity. Users may optionally save simulation results to Firestore for future reference.

2.2.11 AI Budget Advisor Module

The AI Budget Advisor provides personalized budgeting guidance grounded in the well established 50/30/20 personal finance rule, making actionable financial advice accessible to users who cannot afford professional financial advisory services. The module implements a three step wizard collecting monthly income and categorized expense amounts, then applies the rule locally to classify spending into needs (50%), wants (30%), and savings (20%) targets. Deviations from target allocations are computed per category, ranked by magnitude, and translated into up to five plain language advice items expressed without financial jargon in the user's preferred language. When internet connectivity is available, the system optionally invokes a Gemini Cloud Function for enhanced, contextually richer advice. Budget plans and advice summaries are persisted to Firestore for historical review and longitudinal comparison across budgeting sessions.

2.2.12 Rewards and Gamification Module

The Rewards and Gamification Module applies behavioral science principles to sustain user engagement and reinforce positive financial behaviors over time. Users earn reward points for completing quizzes with a score of 80% or higher and for reaching their savings goal targets, creating a direct incentive link between learning, saving, and reward accumulation . A configurable daily points cap of 50 points per day prevents artificial point inflation while sustaining genuine daily engagement. The daily counter resets at midnight based on a date comparison in RewardsService(). Badges are awarded for defined milestones first lesson completed, first savings goal created, five lessons completed, and first goal reached providing recognition markers that encourage progressive exploration of platform features. All reward updates use atomic Firestore transactions to prevent inconsistencies from concurrent updates. The Rewards screen displays the current points balance and all earned badges with their acquisition dates.

2.2.13 Community Forums Module

The Community Forums Module provides a peer to peer knowledge sharing and mutual support environment within FinEase, allowing users to discuss financial topics, share savings strategies, ask questions about NGO programs, and offer community based encouragement. The module organizes discussions into topic categories aligned with FinEase's core financial domains, enabling users to discover relevant threads quickly. Users can create new discussion threads and post replies to existing threads, with thread post counts updated atomically through Firestore to

maintain accurate engagement metrics. Administrators retain moderation authority to remove content that violates community guidelines, ensuring the forum remains a constructive and safe environment. All forum content is stored in Firestore under per thread document structures, with real time listeners providing immediate visibility of new replies without requiring page refresh. The social proof dimension of community forums seeing peers actively saving, learning, and seeking welfare support reinforces individual motivation and normalizes engagement with formal financial tools among users who may otherwise feel isolated in their financial challenges.

2.2.14 Data Management and Storage

FinEase uses Google Firebase Cloud Firestore as its exclusive primary data store, organizing all system data across fourteen top level collections. The data management strategy is governed by four principles: each domain entity maps to a dedicated collection; the Firebase Authentication userId serves as the universal foreign key linking all user owned documents; frequently co accessed data is denormalised by embedding it within parent documents (e.g. quiz questions within quiz documents, expense breakdowns within budget plan documents) to minimize read operations; and infrequently changing reference data such as lesson content and NGO listings is cached client side through Firestore's offline persistence feature . All multi document writes use Firestore atomic transactions to guarantee data consistency under concurrent access. Firebase Security Rules provide an independent database level access control layer that enforces per user data isolation and admin only paths regardless of client application behavior. Uploaded files are referenced by Firestore document fields and stored in Firebase Storage where applicable. The data design is documented comprehensively in the Data Dictionary.

2.2.15 External API and Service Integration

FinEase integrates with five distinct Firebase and Google services through well defined API contracts. Firebase Authentication provides signInWithCredential APIs for email/password based user registration and login. Cloud Firestore provides the collection, runTransaction, and onSnapshot APIs for all persistent data operations. Firebase Cloud Functions provide HTTPS Callable endpoints for the AI Chatbot and enhanced Budget Advisor, acting as secure server side proxies that shield the Gemini API key from exposure in the client application. Firebase Cloud Messaging provides the Admin SDK send API, invoked exclusively from Cloud Functions, for savings reminders. The Gemini API provides the generateContent endpoint for bilingual

conversational AI responses and enhanced budget advice. All external integrations use HTTPS with TLS 1.2 or higher and all sensitive credentials are stored as Cloud Function secrets rather than in the client codebase. The system architecture maintains clean separation between client facing APIs and external service integrations through the Application/Service Layer.

2.2.16 Security and Privacy

Security and privacy are first class requirements in FinEase given that the system stores sensitive personal and financial information about financially vulnerable users. All data transmission between the client and Firebase services uses HTTPS with TLS 1.2 or higher, preventing interception of authentication tokens and financial data in transit. Firebase Security Rules enforce per user data isolation at the database level, ensuring that no user can read or write another user's Firestore documents regardless of the client application's behavior. Admin only Firestore paths are protected by Custom Claims checks in Security Rules, so even a compromised regular user account cannot access administrative data. The Gemini API key is stored exclusively as a Firebase Cloud Function secret using the firebase functions:secrets:set command and is never embedded in the client APK or repository. A comprehensive automated Security Rules test suite of eighteen test cases verifies all access control policies across all fourteen Firestore collections, providing formal assurance rather than reliance on manual inspection alone.

2.2.17 Scalability and Performance Optimization

FinEase is architected to remain performant and scalable as the user base and data volume grow, leveraging Firebase's serverless infrastructure which scales automatically without manual capacity planning. Specific performance targets are defined for all primary operations: dashboard above fold content must load within three seconds on 4G, Firestore reads must complete and render within two seconds, the AI chatbot must respond within eight seconds, and the client side loan simulation must complete within 500 milliseconds . Firebase Cloud Firestore's offline persistence eliminates round trip latency for previously accessed content, and Provider based reactive state management ensures that UI updates are driven by Firestore real time listeners rather than polling, minimizing both latency and unnecessary network traffic. Denormalization of frequently co accessed data reduces read counts per screen load. Firebase Cloud Functions scale automatically on demand, ensuring that AI chatbot and other workloads do not create contention with core

application data operations. The modular multi tier architecture ensures that individual service components can be updated or replaced independently without impacting the full system.

2.2.18 Documentation

Comprehensive documentation is produced at every phase of the FinEase development lifecycle, ensuring academic rigor, maintainability, and transparency of design decisions. The Scope Document defined the project boundaries, objectives, and incremental delivery plan. The Software Requirements Specification (SRS), provides a complete, traceable record of all functional requirements, non functional requirements, use cases, system actors, and constraints. The Software Design Description (SDD), documents the multi tier system architecture, all UML design models, the Firestore data dictionary, and all algorithms. This Final Year Project Report provides the full narrative of the project lifecycle. The GitHub repository is maintained with meaningful commit messages that document incremental progress. All design artifacts use case diagrams, activity diagrams, class diagrams, sequence diagrams, and state transition diagrams are embedded in the report with accompanying descriptive text. In code documentation follows Dart doc comment conventions for all public methods and classes.

2.2.19 Testing and Quality Assurance

FinEase is subjected to a comprehensive, multi level testing strategy following the V model approach, ensuring that requirements, design, and implementation are all verified through corresponding test activities. Unit testing covers all core algorithms with 23 test cases verifying loan simulation, quiz scoring and rewards logic, savings goal progress updates, and budget advice generation. Functional testing validates all use case flows against their stated requirements, covering user authentication, savings goals, NGO directory, loan simulation, and the AI chatbot. Integration testing verifies eleven cross module interaction scenarios including quiz to rewards, goal to reward, and chatbot to Firestore flows. The Firebase Emulator Suite enables automated integration testing with mock Firebase services. Eighteen Firebase Security Rules test cases provide formal access control verification. All 61 automated test cases pass at 100%. Requirements traceability from every functional and non functional requirement to its test cases is maintained in the traceability matrix.

2.2.20 User Training and Support Materials

Given that FinEase's target users may have limited digital literacy and no prior experience with financial applications, user training and support materials are an integral deliverable of the project. The application itself incorporates contextual guidance at the point of need: onboarding screens introduced during first launch explain the application's core features; quick action suggestion chips on the chatbot screen provide example queries for first time users; inline validation messages guide correct form completion; and confirmation dialogs before irreversible actions (such as goal cancellation) prevent accidental data loss. The bilingual Urdu/English interface is itself a support mechanism, enabling users to interact in their primary literacy language. Administrator support materials include guidance for managing NGO listings, reviewing applications, and understanding the admin analytics dashboard. The project report provides a complete user workflow description in Chapter 5, and the detailed use case descriptions serve as a reference for understanding system behavior from the user's perspective.

2.2.21 Maintenance, Updates, and Continuous Improvement

The long term sustainability of FinEase is supported by an architecture and codebase designed for ease of maintenance, incremental improvement, and modular extension. The feature first Flutter project structure isolates each module's code, ensuring that changes to one module do not inadvertently affect others. Service class singletons and repository abstractions provide clean interfaces that can be reimplemented or extended without modifying dependent code. Firebase's serverless infrastructure eliminates server maintenance overhead, and Firebase Security Rules can be updated and deployed independently of the client application. The comprehensive automated test suite enables regression detection during maintenance updates, providing confidence that changes do not break existing functionality. The modular design explicitly supports the future work enhancements, including real transaction integration, regional language expansion, iOS release, and machine learning personalization without requiring architectural restructuring. The requirements traceability matrix ensures that any future change to requirements can be traced to its affected design and test components, supporting systematic impact analysis during maintenance cycles.

2.2.22 Hardware and Software Requirements

The system requires specific hardware and software configurations to ensure smooth development and testing. For hardware, the development machine should have at minimum an Intel i5 processor with 8 GB RAM and 20 GB disk space, though an Intel i7 with 16 GB RAM and SSD storage is recommended for optimal performance. The test device must run Android 8.0 or higher with at least 2 GB RAM and a 3G/4G connection, with Android 11+ and 4 GB RAM being the preferred setup. A stable internet connection of at least 10 Mbps is required for development, with 25 Mbps broadband recommended. On the software side, the project utilizes Flutter SDK (version 3.x or higher) as the cross-platform mobile development framework, with Dart bundled alongside it as the primary programming language. Android Studio (Electric Eel or higher) serves as the main IDE with Android toolchain support, while Firebase CLI (latest stable) handles project management and deployment. Node.js (version 18.x LTS) is also required as the runtime environment for Firebase Cloud Functions

2.3 Current System Analysis

A thorough understanding of existing systems in the financial inclusion space is essential to justify FinEase's design decisions and identify the gaps it fills. The following provides a critical analysis of the most relevant existing systems, summarized in Table 2.1.

2.3.1 JazzCash and Easypaisa

JazzCash and Easypaisa are the two dominant mobile financial service providers in Pakistan, collectively serving over 100 million registered accounts. Both offer money transfers, bill payments, and merchant payments. Critical limitations relative to FinEase: neither provides financial literacy education, savings goal tracking with reminders and analytics, NGO program discovery, loan simulation tools, budget advisory, or AI powered guidance. Both are primarily English language, do not support gamification, and are designed for transactional rather than financial wellness use. application guidance; neither platform consolidates information about the full range of available welfare programs; and neither offers financial education or management tools.

2.3.2 Comparison Summary

Table 2.1: Comparison of Existing Systems with FinEase

Feature	JazzCash / Easypaisa	Govt. Portals	Intl. Apps	FinEase
Financial Literacy Education	No	No	Partial	Yes
Savings Goal Tracking	No	No	Yes	Yes
Loan Simulation Tool	No	No	Yes	Yes
NGO / Welfare Discovery	No	Partial	No	Yes
AI Budget Advisory	No	No	Partial	Yes
Bilingual Urdu/English	Partial	Partial	No	Yes
Offline Functionality	No	No	Partial	Yes
Gamification / Rewards	No	No	Partial	Yes
AI Chatbot Support	No	No	Partial	Yes
Free to Use	Yes	Yes	No	Yes
Designed for Pakistan	Yes	Yes	No	Yes
No Bank Account Required	Yes	Yes	No	Yes

Table 2.1 compares FinEase against five existing financial applications across key dimensions including bilingual support, AI features, offline access, NGO integration, and gamification, highlighting the gaps FinEase addresses.

Chapter 3

Requirement Analysis

3 Requirement Analysis

Requirement analysis for FinEase was carried out to identify all actors, workflows, constraints, and system behaviors needed to build a correct, secure, and useful financial inclusion platform. The approved SRS uses UML use case diagrams, as mentioned in Figure 3.1 and Figure 3.2, and detailed use case descriptions, as mentioned in Table 3.1 through Table 3.9, to represent interactions among the User and the Admin actor, while defining functional requirements and non functional requirements. All requirements are traced to design components and test cases in the requirements traceability matrix.

3.1 System Actors

The system defines two primary actors: the User and the Admin, each with distinct roles and responsibilities. The User is an individual who has successfully completed authentication and is granted access to all FinEase modules, including financial literacy, savings goals, NGO directory, loan simulation, AI chatbot, budget advisor, and rewards. In contrast, the Admin is an authenticated user with elevated privileges, identified through Firebase Custom Claims, and is responsible for overseeing and managing the system. Administrative tasks include approving or rejecting NGO listings, moderating forums, managing user accounts, viewing analytics, and maintaining lesson content. This distinction clearly outlines the different levels of access and responsibilities assigned to each actor within the system.

3.2 Use Case Diagrams

The FinEase case model comprises two primary diagrams: one for User i.e. Figure 3.1, and one for Admin i.e. Figure 3.2. These diagrams identify all major system use cases and their include/extend relationships. They provided the functional basis for the design models, implementation, and testing.

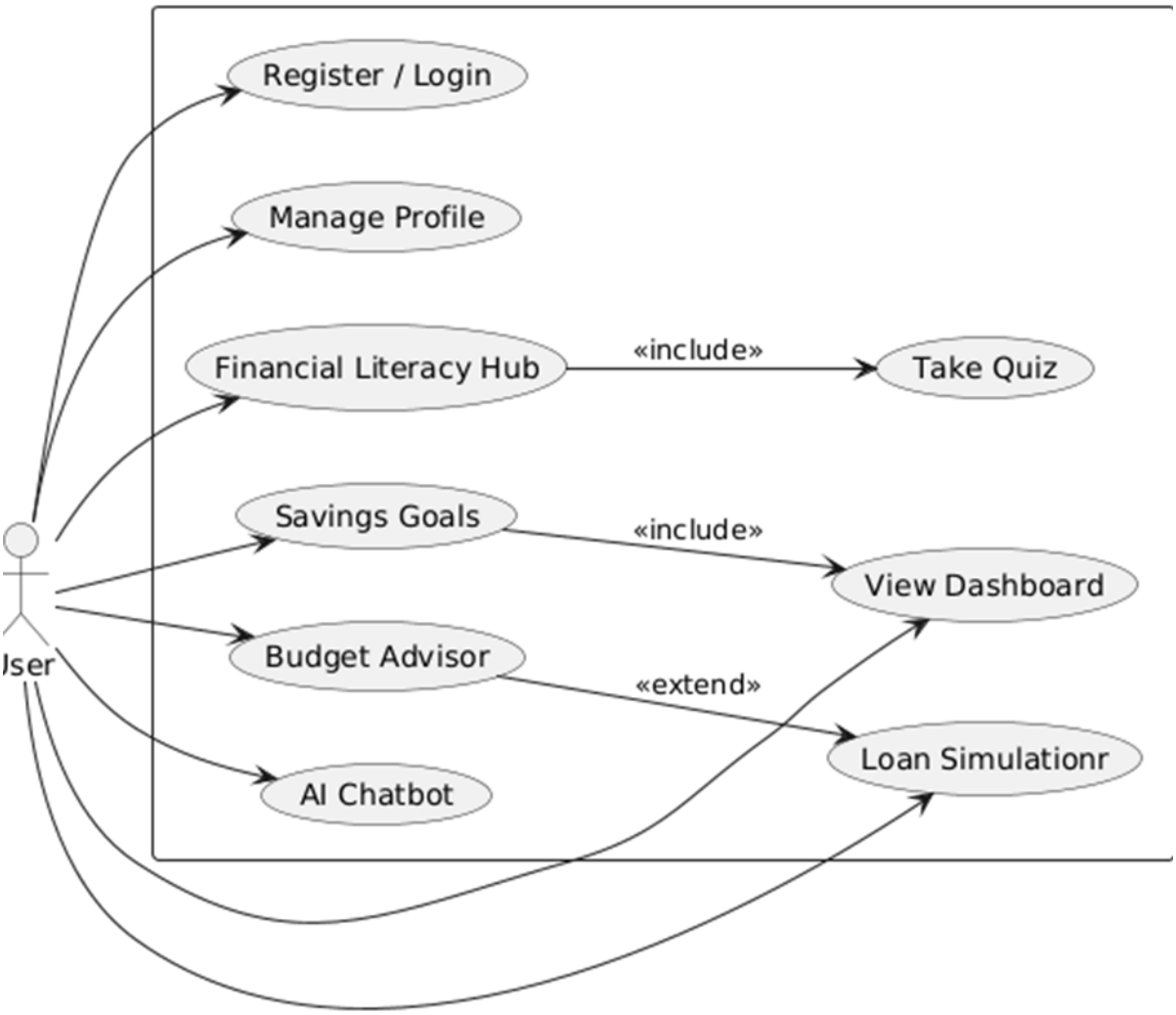

Figure 3.1: Use Case Diagram: User and Admin

Figure 3.1 represents the interaction between the user and the system through various functionalities. It shows that the user can perform actions such as registration/login, profile management, accessing the financial literacy hub, setting savings goals, using the budget advisor, and interacting with the AI chatbot. Some use cases are connected with «include» relationships, such as taking quizzes within the financial literacy module and viewing the dashboard from savings goals, while «extend» relationships highlight optional features like loan simulation linked

with the budget advisor. Overall, the diagram illustrates how different system features are interconnected and how the user interacts with them in a structured way.

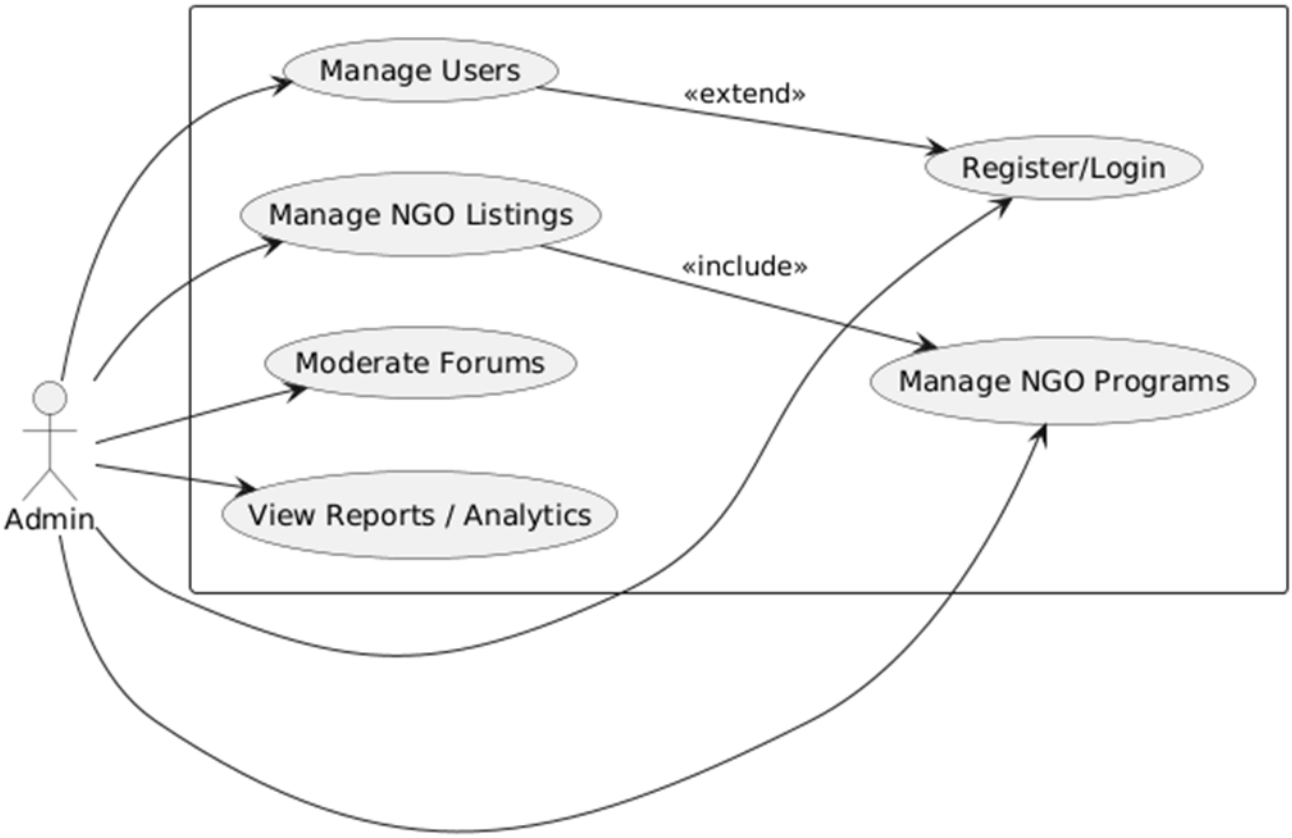

Figure 3.2: Use Case Diagram: Admin

Figure 3.2 presents the Admin use case diagram showing administrator-exclusive interactions including NGO program listing, community forum moderation, and platform analytics access.

This Use Case Diagram shows the administrative use cases available to the Admin actor, including managing NGO program listings, reviewing and updating application statuses, moderating community forum content, and monitoring platform analytics.

3.3 Detailed Use Case Descriptions

The following tables define the interactions for each major module of FinEase. They form the direct basis for functional requirements in Section 3.4, activity diagrams in Section 4.2, sequence diagrams in Section 4.4, and functional test cases in Section 6.3.

UC 01: User Registration

Table 3.1: Use Case Description: User Registration (UC 01)

Use Case ID	UC 01
Use Case Name	User Registration
Actor(s)	User, Admin
Description	A new user creates a FinEase account email address.
Preconditions	PRE 1: User is unauthenticated. PRE 2: Internet available. PRE 3: User has a valid email.
Basic Flow	1. User selects Register. 2. User chooses registration method. 3. User enters details (name, contact, language). 4. System validates with different validation techniques. 5. System creates user document in Firestore. 6. System navigates to Dashboard.
Alternative Flows	A: Incorrect email address; shows error message. B: Email already registered error: Account exists. Please login.
Postconditions	POST 1: Firebase Auth account created. POST 2: Firestore users document created.

Table 3.1 specifies the email registration flows, preconditions, normal and alternative flows, postconditions, and exception handling for duplicate accounts.

UC 02: Financial Literacy Hub

Table 3.2: Use Case Description: Financial Literacy Hub (UC 02)

Use Case ID	UC 02
Use Case Name	Access Financial Literacy Hub
Actor(s)	User
Description	User accesses educational financial lessons organized by topic and difficulty, then attempts quizzes to earn reward points.
Preconditions	PRE 1: User is authenticated. PRE 2: At least one lesson published in Firestore.
Basic Flow	1. User taps Learn tab. 2. LiteracyService fetches lessons filtered by language. 3. User selects a lesson. 4. System renders Markdown content with correct text direction. 5. User proceeds to quiz. 6. QuizService evaluates score against passThreshold (80%). 7. If passed: RewardsService awards points. 8. System displays score and feedback.
Alternative Flows	A: No internet cached lesson content shown offline. B: Score below 80% 0 points; retry available.
Postconditions	POST 1: QuizAttempt stored in Firestore. POST 2: Rewards updated if passed.

Table 3.2 details lesson browsing, category filtering, Markdown rendering, quiz scoring against the 80% threshold, and conditional reward point award upon passing.

UC 03: Savings Goal Tracker

Table 3.3: Use Case Description: Savings Goal Tracker (UC 03)

Use Case ID	UC 03
Use Case Name	Manage Savings Goals
Actor(s)	User

Description	User creates, tracks, deposits toward, and completes personalized savings goals.
Preconditions	PRE 1: User is authenticated.
Basic Flow	1. User navigates to Save tab. 2. User taps Create New Goal. 3. User enters title, PKR target, target date. 4. SavingsService.save() called; FCM reminder scheduled. 5. Goal shows at 0% progress. 6. User records a deposit. 7. Firestore.runTransaction() atomically updates currentAmount, progress, status. 8. If target reached: Completed status; RewardsService.award() called.
Alternative Flows	A: User cancels goal status set to Cancelled. B: Deposit exceeds remaining balance capped at target.
Postconditions	POST 1: SavingsGoal document updated.

Table 3.3 covers savings goal creation, atomic deposit recording via Firestore transactions, progress calculation, automatic goal completion, and FCM reminder scheduling one day before the target date.

UC 04: NGO Directory

Table 3.4: Use Case Description: NGO Directory (UC 04)

Use Case ID	UC 04
Use Case Name	Browse to NGO Programs
Actor(s)	User, Admin
Description	User searches and filters welfare programs and submits applications. Admin manages listings.
Preconditions	PRE 1: User is authenticated. PRE 2: At least one approved NGO program exists in Firestore.

Basic Flow	1. User opens Support tab. 2. System displays all approved programs. 3. User applies category filter. 4. User selects a program. 5. User taps Apply Now. 6. System opens NGO Website. 7. Admin manages NGO listings.
Postconditions	POST 1: NGOApplication in Firestore with Submitted status. POST 2: User can track status.

Table 3.4 describes the searchable welfare program listing, category filtering, program detail view, application submission with draft saves, and status tracking for submitted applications.

UC 05: Loan Simulation

Table 3.5: Use Case Description: Loan Simulation (UC 05)

Use Case ID	UC 05
Use Case Name	Simulate Loan Repayment
Actor(s)	User
Description	User inputs loan parameters to receive a detailed repayment breakdown enabling informed borrowing decisions.
Preconditions	PRE 1: User is authenticated. No internet required calculation is client side.
Basic Flow	1. User accesses Loan Simulation. 2. User enters principal, annual interest rate (%), and tenure (months). 3. LoanSimulationService.calculate() executes amortization algorithm. 4. System displays monthly installment, total repayable, and total interest in PKR. 5. Affordability indicator shown. 6. User optionally saves result.
Postconditions	POST 1: LoanSimulation document optionally stored.

Table 3.5 defines inputs for the loan amortization calculator, computation of monthly installment, total repayable amount, total interest, affordability indicator display, and optional result save to Firestore.

UC 06: AI Budget Advisor

Table 3.6: Use Case Description: AI Budget Advisor (UC 06)

Use Case ID	UC 06
Use Case Name	Get AI Budget Advice
Actor(s)	User
Description	User provides income and expense data to receive personalised AI generated budget advice based on the 50/30/20 rule.
Preconditions	PRE 1: User is authenticated.
Basic Flow	1. User selects Budget Advisor. 2. Step 1: Enter monthly income (PKR). 3. Step 2: Enter expense breakdown. 4. BudgetAdvisorService applies 50/30/20 rule locally. 5. Up to 5 ranked advice items generated. 6. If internet available: Gemini Cloud Function invoked. 7. BudgetPlan saved to Firestore.
Postconditions	POST 1: BudgetPlan document stored.

Table 3.6 specifies the three-step income and expense entry wizard, local 50/30/20 rule analysis, optional Gemini-enhanced advice via Cloud Function, and budget plan persistence to Firestore.

UC 07: AI Chatbot

Table 3.7: Use Case Description: AI Chatbot (UC 07)

Use Case ID	UC 07
Use Case Name	Interact with AI Chatbot
Actor(s)	User
Description	User engages in bilingual conversational interaction with the Gemini powered chatbot for financial guidance, NGO info, or app navigation.

Preconditions	PRE 1: User is authenticated. PRE 2: Internet required.
Basic Flow	1. User opens Chatbot screen. 2. User types query in Urdu or English. 3. ChatbotService sends messages and last 10 message pairs to Cloud Function. 4. Cloud Function enriches prompt and calls Gemini API. 5. Response returned in user's language. 6. Exchange stored in ChatbotSession Firestore document.
Alternative Flows	A: Gemini unavailable graceful error message with fallback FAQ links.
Postconditions	POST 1: Conversation history updated.

Table 3.7 details the conversational interface with Gemini API integration, bilingual query handling, ten-message context window, typing indicator, quick-action chips, and graceful offline error fallback.

UC 08: Community Forums

Table 3.8: Use Case Description: Community Forums (UC 08)

Use Case ID	UC 08
Use Case Name	Participate in Community Forums
Actor(s)	User, Admin
Description	Users create and participate in community forum discussions organised by financial topic category. Admin moderates' content.
Basic Flow	1. User opens Community tab. 2. System displays thread list by category. 3. User creates a new thread. 4. User posts a reply. 5. ForumPost saved; thread postCount incremented atomically.
Postconditions	POST 1: ForumThread/ForumPost documents updated in Firestore.

Table 3.8 describes peer discussion thread creation, reply posting, category-based organization, thread engagement metrics, and admin moderation authority for content removal.

UC 09: Rewards and Gamification

Table 3.9: Use Case Description: Rewards and Gamification (UC 09)

Use Case ID	UC 09
Use Case Name	Earn and View Rewards
Actor(s)	User
Description	Users earn points and badges by completing quizzes and savings goals. A daily cap prevents gaming while sustaining genuine engagement.
Basic Flow	1. User passes a quiz (score >= 80%). 2. RewardsService.awardQuizPoints() called. 3. System checks daily earned points against DAILY_CAP (50 pts). 4. Points awarded up to remaining daily allowance. 5. Badge conditions checked. 6. Rewards screen updates reactively.
Postconditions	POST 1: Rewards document updated in Firestore.

Table 3.9 covers the points award mechanism for quiz passes and goal completion, daily cap enforcement, badge unlock conditions, and midnight reset of the daily points counter.

3.4 Functional Requirements

All functional requirements of FinEase are organized by module below. Each traces to a use case, design component, and test case as documented in the requirements traceability matrix.

3.4.1 User Authentication (FR UA)

Table 3.10: Functional Requirements: User Authentication (FR UA)

Req. ID	Requirement Description	Priority
FR UA 01	The system shall allow new users to register using an email address.	High

Req. ID	Requirement Description	Priority
FR UA 02	The system shall allow users to log in using their email.	High
FR UA 03	The system shall maintain user sessions across application restarts using Firebase Authentication persistent tokens.	High
FR UA 04	The system shall support role based access control with user and admin roles, enforced via Firebase Custom Claims.	High
FR UA 05	The system shall allow users to update their profile: name, preferred language, income estimate, and risk level.	Medium
FR UA 06	The system shall allow users to log out, invalidating the local session token and returning it to the login screen.	High

Table 3.10 lists all authentication requirements including Email OTP registration, login, persistent sessions, role-based access control, language preference, and logout.

3.4.2 Financial Literacy Hub (FR FL)

Table 3.11: Functional Requirements: Financial Literacy Hub (FR FL)

Req. ID	Requirement Description	Priority
FR FL 01	The system shall provide financial literacy lessons categorised by topic and difficulty level (Beginner, Intermediate, Advanced).	High
FR FL 02	Lessons shall be available in both Urdu (RTL) and English (LTR) based on the user's language preference.	High
FR FL 03	The system shall include at least one quiz per lesson with a minimum of 5 questions.	High
FR FL 04	The system shall evaluate quiz submissions and compute a percentage score.	High

Req. ID	Requirement Description	Priority
FR FL 05	The system shall award configured points to users scoring at or above 80% passThreshold using a Firestore transaction.	High
FR FL 06	The system shall track and display lesson completion progress per user (unlocked, in progress, completed).	Medium
FR FL 07	The system shall support offline viewing of previously accessed lesson content via Firestore offline persistence.	Medium

Table 3.11 enumerates requirements for categorized lesson access, bilingual content, quiz evaluation, reward points award, lesson progress tracking, and offline content availability.

3.4.3 Savings Goal Tracker (FR SG)

Table 3.12 Functional Requirements: Savings Goal Tracker (FR SG)

Req. ID	Requirement Description	Priority
FR SG 01	The system shall allow users to create savings goals with a title, target amount (PKR), and target date.	High
FR SG 02	The system shall allow users to record deposits against an active savings goal using a Firestore atomic transaction.	High
FR SG 03	The system shall compute and persist progress percentage after each deposit.	High
FR SG 04	The system shall automatically set goal status to Completed when currentAmount reaches targetAmount.	High
FR SG 05	The system shall allow users to cancel an active savings goal; cancelled goals move to history.	Medium
FR SG 06	The system shall prevent creation of goals with a target date in the past or target amount of zero.	High

Table 3.12 specifies requirements for goal creation, atomic deposit recording, progress calculation, automatic completion detection, FCM reminders, goal cancellation, and input validation.

3.4.4 NGO Directory (FR NG)

Table 3.13 Functional Requirements: NGO Directory (FR NG)

Req. ID	Requirement Description	Priority
FR NG 01	The system shall maintain a directory of NGO programs with name, category, eligibility criteria, and application steps.	High
FR NG 02	Users shall be able to filter programs by category (Financial Aid, Health, Education, Housing, Food Security).	High
FR NG 03	Users shall be able to search programs by name keyword.	High
FR NG 04	Admin shall be able to add, edit, approve, and remove NGO program listings.	High

Table 3.13 defines requirements for program listing, category filtering, keyword search, application submission, status tracking and admin review controls.

3.4.5 Loan Simulation (FR LS)

Table 3.14 Functional Requirements: Loan Simulation (FR LS)

Req. ID	Requirement Description	Priority
FR LS 01	The system shall accept principal, annual interest rate, and tenure in months with inline validation.	High
FR LS 02	The system shall calculate monthly installment using the standard amortization formula M = P*r(1+r)^n / ((1+r)^n 1).	High
FR LS 03	The system shall display total repayable amount and total interest, both formatted in PKR.	High
FR LS 04	The system shall handle the zero interest rate edge case using M = P / n.	High

Req. ID	Requirement Description	Priority
FR LS 05	The system shall display a green/amber/red affordability indicator comparing monthly installment against user income estimate.	Medium
FR LS 06	The system shall allow users to optionally save simulation results to Firestore.	Low

Table 3.14 specifies the amortization formula requirements, PKR formatting, zero-interest edge case, affordability indicator, optional result save, and input validation rules.

3.4.6 AI Budget Advisor (FR BA)

Table 3.15 Functional Requirements: AI Budget Advisor (FR BA)

Req. ID	Requirement Description	Priority
FR BA 01	The system shall allow users to input monthly income and categorize expense amounts across at least 6 categories.	High
FR BA 02	The system shall apply the 50/30/20 rule to classify expenses and compute deviations.	High
FR BA 03	Advice shall be in plain, accessible language without financial jargon, in the user's preferred language.	High
FR BA 04	The system shall limit generated advice to a maximum of 5 items, ranked by magnitude of overspend.	Medium
FR BA 05	The system shall save budget plans and advice to Firestore for historical review.	Medium
FR BA 06	The system shall optionally invoke the Gemini API via Cloud Function for enhanced advice when internet is available.	Low

Table 3.15 lists requirements for income and expense entry, 50/30/20 analysis, advice ranking, five-item cap, budget plan storage, and optional Gemini-enhanced advice.

3.4.7 AI Chatbot (FR CH)

Table 3.16 Functional Requirements: AI Chatbot (FR CH)

Req. ID	Requirement Description	Priority
FR CH 01	The system shall provide a conversational AI chatbot powered by the Gemini API via Firebase Cloud Functions.	High
FR CH 02	The chatbot shall accept and respond to queries in both Urdu and English based on user language preference.	High
FR CH 03	The chatbot shall handle queries about FinEase features, financial guidance, NGO programs, and loan concepts.	High
FR CH 04	The system shall display a typing indicator while awaiting the AI response.	Medium
FR CH 05	The system shall store conversation history in Firestore per session (max 10 message pairs).	Medium
FR CH 06	The system shall display a graceful, user friendly error message if the Gemini API is unavailable.	High
FR CH 07	The system shall provide quick action suggestion chips on the chatbot screen when no conversation history exists.	Low

Table 3.16 enumerates chatbot requirements including Gemini integration, bilingual responses, financial topic scope, typing indicator, conversation history, graceful error handling, and quick-action chips.

3.4.8 Rewards and Gamification (FR RW)

Table 3.17 Functional Requirements: Rewards and Gamification (FR RW)

Req. ID	Requirement Description	Priority
FR RW 01	The system shall award points for quiz passes and savings goal completions using atomic Firestore transactions.	High
FR RW 02	The system shall enforce a configurable daily points cap (default: 50 pts/day) to prevent gaming.	High
FR RW 03	The system shall display current balance points and list of earned badges with acquisition dates.	High
FR RW 04	The system shall award define badges for key milestones: first lesson, first goal, five lessons, first completed goal.	Medium
FR RW 05	The daily points counter shall automatically reset at midnight when the user next opens the application.	High

Table 3.17 defines requirements for points award events, daily cap enforcement, badge unlock conditions, leaderboard display, and daily counter reset at midnight.

3.5 Non Functional Requirements

3.5.1 Usability (NFR US)

Table 3.18 Non-Functional Requirements: Usability (NFR US)

Req. ID	Requirement	Measure
NFR US 01	All UI components shall comply with Material Design 3 guidelines.	Design review checklist
NFR US 02	All primary user tasks shall be completable within 3 taps from the main dashboard.	User journey analysis

Req. ID	Requirement	Measure
NFR US 03	The application shall provide full Urdu and English interface with correct RTL text direction for Urdu.	Language switch test
NFR US 04	Minimum font size for body text shall be 14sp; heading text 18sp. Contrast ratio >= 4.5:1.	UI inspection
NFR US 05	All interactive tap targets shall have a minimum touchable area of 48x48 dp.	Accessibility audit
NFR US 06	All form fields shall display inline validation errors within 500ms of invalid input.	Manual testing with stopwatch

Table 3.18 specifies usability requirements including three-tap navigation, bilingual RTL support, language switching, contrast ratios, minimum tap target size, and inline validation feedback timing.

3.5.2 Performance, Security, Scalability and Compatibility

Table 3.19 Non-Functional Requirements: Performance, Security, Scalability, and Compatibility

Req. ID	Requirement	Measure
NFR PF 01	Dashboard shall load all above fold content within 3 seconds on 4G.	Stopwatch on physical device
NFR PF 02	Firestore reads shall complete and render within 2 seconds.	Firebase Performance Monitoring
NFR PF 03	AI chatbot shall display a response within 8 seconds on 4G.	Stopwatch measurement
NFR PF 04	Loan simulation shall be completed within 500ms (client side operation).	Dart performance profiler

Req. ID	Requirement	Measure
NFR SE 01	All data transmission shall use HTTPS/TLS 1.2 or higher.	Network traffic inspection
NFR SE 02	Firebase Security Rules shall enforce per user Firestore data isolation.	Security Rules automated test suite
NFR SE 03	The Gemini API key shall never be in the client application; stored as Cloud Function secret only.	Code review and APK inspection
NFR SE 04	Admin only Firestore paths shall be inaccessible to user role accounts.	Security Rules test suite
NFR RL 01	Core user data shall be accessible offline via Firestore offline persistence.	Airplane mode testing
NFR RL 02	The app shall not crash due to network unavailability; graceful degradation required.	Network interruption testing
NFR RL 03	All multi document Firestore writes shall use transactions for data consistency.	Code review and concurrent tests
NFR PT 01	The application shall run on Android 8.0 (API level 26) or higher.	Device compatibility matrix
NFR PT 02	The application shall function on devices with minimum 2 GB RAM.	Low spec device testing

Table 3.19 covers performance response-time targets, security requirements for data isolation and API key management, scalability thresholds, maintainability guidelines, device compatibility, and communication interface specifications.

3.5.3 Scalability

Scalability is an important non-functional requirement for FinEase because the system is designed to serve a large and growing number of users across Pakistan. To support this growth, FinEase

uses Firebase, which provides a serverless and automatically scalable cloud infrastructure. Cloud Firestore can handle a large number of users by scaling automatically, allowing many data reads and writes at the same time. Firebase Authentication also manages user logins efficiently without needing manual setup. Firebase Cloud Functions automatically adjust based on demand, ensuring that features like chatbots and recommendation systems continue to work smoothly even during high usage. On the user side, the Flutter app is designed so that each user mainly interacts with their own data, which helps maintain good performance even as the number of users increases. The database structure is optimized to reduce unnecessary data loading, improving speed and lowering costs. Finally, the system is modular, meaning different parts can be updated or improved without affecting the entire system, making it easier to handle growth over time.

3.5.4 Maintainability

Maintainability is addressed through deliberate architectural and coding conventions that minimize the effort required to understand, modify, test, and extend FinEase over its operational lifetime. The feature first Flutter project structure groups all files related to a specific module screens, widgets, and utilities within that module's directory, making it straightforward to locate and modify any module in isolation. The strict three layer OOD architecture separates domain objects (data), service classes (business logic), and repository classes (data access), ensuring that changes to data storage technology, business rules, or UI do not cascade across all layers. Repository abstractions define consistent interfaces that can be mocked during testing or replaced with alternative implementations without modifying service or UI code. All business logic resides in service classes rather than in Flutter widget code, making algorithmic changes testable independently of the UI. The comprehensive automated test suite provides regression protection, enabling maintainers to detect whether a change inadvertently breaks existing behavior before deployment. Firebase Security Rules are defined declaratively and versioned in the repository, allowing access control policies to be reviewed, updated, and tested independently of the application code.

3.5.5 Mobile Compatibility

FinEase is designed to work on a wide range of Android devices, especially affordable smartphones commonly used in Pakistan. It supports Android 8.0 and above, which includes most

active devices, ensuring that users with older phones can still use the app. The app runs smoothly even on devices with as little as 2 GB of RAM, making it suitable for entry-level smartphones. Flutter helps provide smooth performance and responsive design, so the app looks and works well on different screen sizes, from small to large devices.

The interface adjusts automatically to different screen sizes, and supports both English and Urdu text, ensuring proper display for all users. Buttons and touch areas are designed to be large enough for easy interaction on all devices. Additionally, FinEase supports offline access through Firestore, allowing users to view previously loaded content even when internet connectivity is weak or unavailable—something that is common in many areas of Pakistan.

3.5.6 Communication Interfaces

FinEase uses several communication methods to connect the mobile app with cloud services and external APIs. The main communication between the app and the database happens through the Firestore SDK, which securely sends and receives data over the internet using encrypted HTTPS connections. This is used for reading, writing, and real-time updates.

User authentication (such as login and signup) is handled through Firebase Authentication, which also uses secure HTTPS connections. For advanced features like the AI Chatbot and Budget Advisor, the app communicates with Firebase Cloud Functions. These functions process requests securely and only work when the user is logged in. When the system needs AI responses, Cloud Functions communicate with external AI APIs (like Gemini) securely, and the API key is stored safely on the server, not in the app.

Finally, the app monitors internet connectivity. If the internet is unavailable, it switches to offline mode using Firestore's offline storage, allowing users to continue accessing previously loaded data.

Chapter 4

Design and Architecture

4 Design and Architecture

The design and architecture of FinEase were developed to satisfy the functional requirements and non functional requirements identified during requirement analysis, while keeping the system modular, secure, scalable, and easy to extend. FinEase is designed as a multi tier client cloud application, as shown in Figure 4.1, with the Presentation, Application/Service, Data Access, and Backend Cloud layers separated into distinct tiers with well defined interfaces. This separation allows independent development, testing, and scaling of each module without affecting the entire system.

4.1 System Architecture

FinEase follows a clean multi tier client cloud architecture. The Presentation Layer consists of Flutter/Dart screens for User and Admin roles. The Application/Service Layer manages business logic through eleven module specific service classes. The Data Access Layer abstracts Firestore through fourteen typed repository classes. The Backend Cloud Layer provides Firebase Authentication, Cloud Firestore, Cloud Functions, Firebase Cloud Messaging, and Gemini API. Firebase Security Rules provide an independent database level security layer.

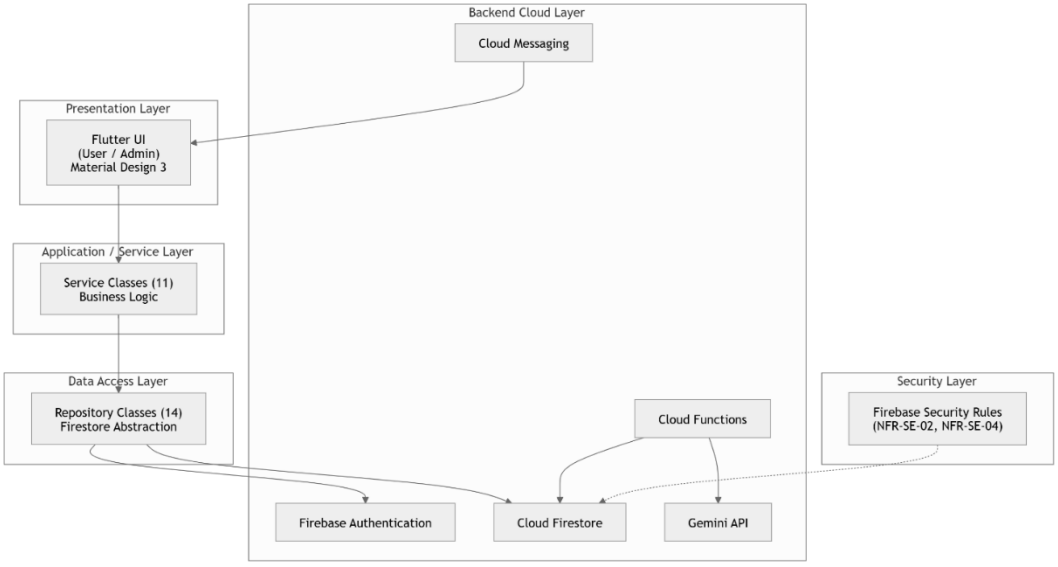

Figure 4.1: FinEase High Level Multi Tier System Architecture

Figure 4.1 illustrates the four tier client cloud architecture comprising the Presentation Layer (Flutter screens), Application/Service Layer (business logic), Data Access Layer (repository classes), and Backend Cloud Layer (Firebase and Gemini services).

4.1.1 Presentation Layer

Built entirely in Flutter/Dart using Material Design 3 components, the Presentation Layer communicates with the Application/Service Layer through dependency injected service instances managed by Provider state management. When Firestore data changes for example, when a savings goal deposit is recorded the relevant service notifies its listeners, causing the UI to rebuild reactively without explicit refresh calls. The Presentation Layer contains no business logic, no direct Firestore queries, and no Firebase API calls.

4.1.2 Application / Service Layer

The business logic core of FinEase contains one service class per module: AuthService, LiteracyService, SavingsService, NGOService, LoanSimulationService, BudgetAdvisorService, CommunityService, RewardsService, PartnerService, AdminService, and ChatbotService. All service classes are instantiated as singletons through Provider, ensuring a single instance is shared across the application lifecycle, preventing duplicate Firestore listeners and maintaining consistent state across screen navigations.

4.1.3 Data Access Layer and Backend Cloud Layer

Repository classes provide a clean abstraction over Firebase's Firestore API, implementing consistent create(), read(), update(), delete(), and list() interfaces that translate between Dart domain objects and Firestore documents. Service classes never write raw Firestore queries they call typed repository methods. The Backend Cloud Layer comprises Firebase Authentication, Cloud Firestore (primary NoSQL database), Cloud Functions (Node.js 18, serverless) and the Gemini API. The Gemini API key is stored exclusively as a Cloud Function secret and never in the client APK.

4.2 Activity Diagrams

The following activity diagrams illustrate the main workflows of FinEase. Each cross references its corresponding use case description and functional requirements.

4.2.1 Financial Literacy Hub Flow

The literacy flow begins when the user selects the Learn tab. LessonRepository queries lessons filtered by language preference. On lesson completion, QuizService evaluates score against passThreshold. If passed, a Firestore transaction atomically awards points while enforcing the daily cap, then checks badge conditions.

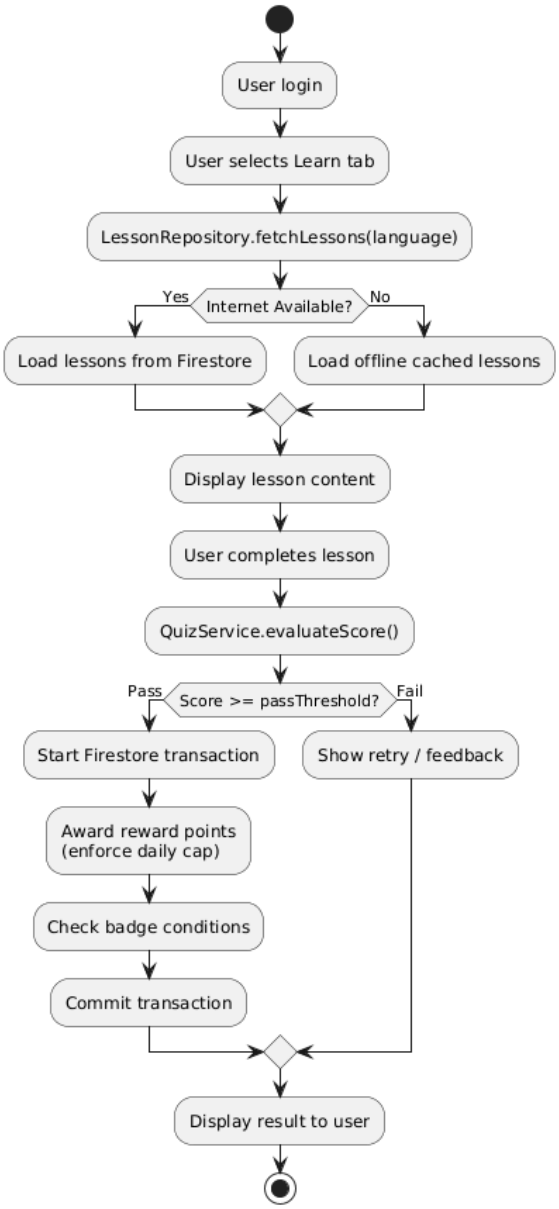

Figure 4.2: Activity Diagram: Financial Literacy Hub Flow

Figure 4.2 shows the step-by-step activity flow for browsing lesson categories, selecting and completing a lesson, attempting the associated quiz, and the conditional award of reward points upon passing.

4.2.2 Savings Goal Management Flow

Goal creation writes a SavingsGoal document and schedules an FCM reminder via a Cloud Function onCreate trigger. Each deposit uses Firestore.runTransaction() for atomic updates: read document, compute new amount (capped at target), compute progress, determine status, write update atomically. On Completed status, RewardsService.awardGoalCompletion() is called.

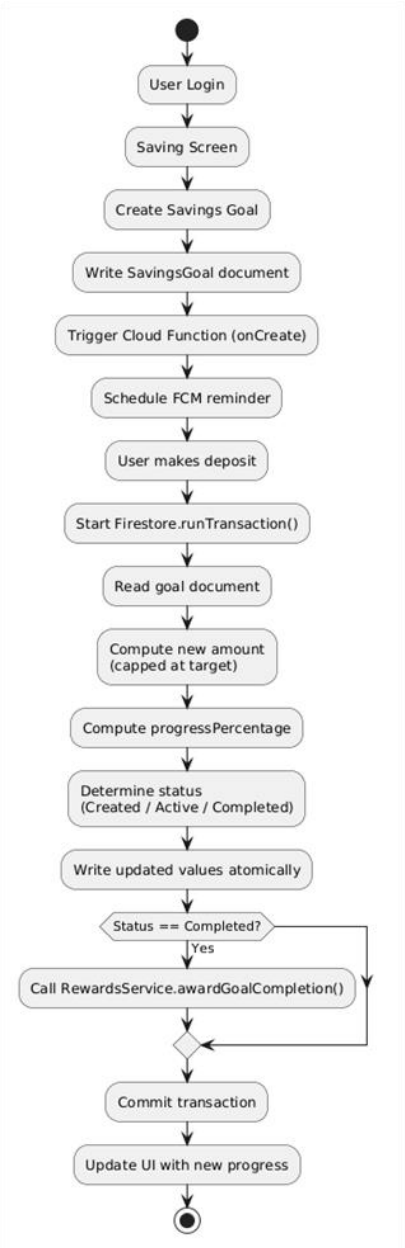

Figure 4.3: Activity Diagram: Savings Goal Management Flow

Figure 4.3 depicts the lifecycle flow for creating a savings goal, recording deposits via Firestore atomic transactions, automatic goal completion when the target is reached, and the associated reward is earned.

4.2.3 Loan Simulation Flow

The loan simulation flow is entirely client side and offline capable. LoanSimulationScreen validates inputs using Flutter's Form widget. LoanSimulationService.calculate() executes the

amortization algorithm, handling the zero interest rate edge case. Results are formatted in PKR. Affordability indicator computes monthly installment as a percentage of the user's income estimate.

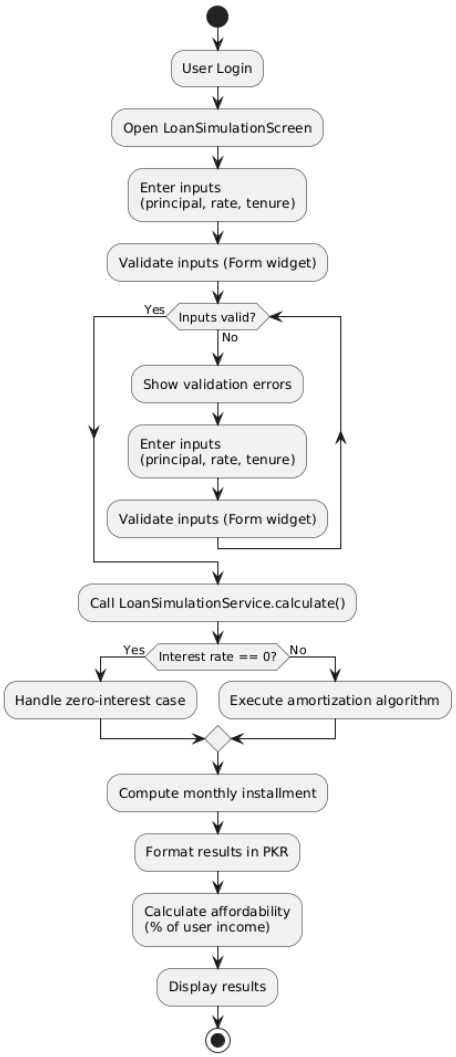

Figure 4.4: Activity Diagram: Loan Simulation Flow

Figure 4.4 outlines the loan simulation process from input of principal, interest rate, and tenure through amortization formula computation to display of monthly installment, total repayable amount, affordability indicator, and optional save to Firestore.

4.2.4 AI Budget Advisor Flow

The Budget Advisor implements a three step wizard. BudgetAdvisorService.generateAdvice() applies the 50/30/20 rule locally, generating up to 5 ranked advice items. If internet is available, an enhanced Gemini Cloud Function is invoked. The BudgetPlan is saved to Firestore.

Figure 4.5: Activity Diagram: AI Budget Advisor Flow

Figure 4.5 shows the two-step budget input flow, application of the 50/30/20 rule, ranked advice generation capped at five items, and storage of the resulting BudgetPlan document in Firestore.

4.2.5 AI Chatbot Flow

The chatbot flow sends the message alongside the last 10 session message pairs to a Firebase HTTPS Callable Cloud Function. The Cloud Function builds a Gemini API request with a FinEase

specific Pakistani finance system prompt, calls the API, and returns the response. The exchange is stored in the ChatbotSession Firestore document. If the API is unavailable, a graceful error is shown.

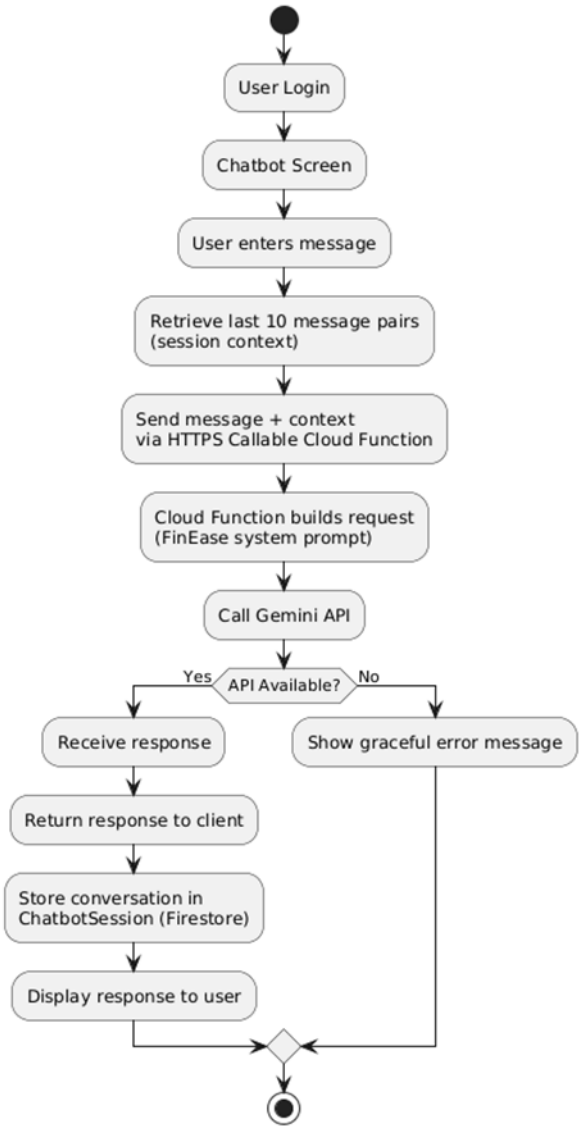

Figure 4.6: Activity Diagram: AI Chatbot Conversation Flow

Figure 4.6 illustrates the bilingual chatbot interaction cycle: user message submission, typing indicator display, Cloud Function invocation with rolling conversation context, Gemini API call, response rendering, and ChatbotSession persistence in Firestore.

This Activity Diagram models the AI Chatbot interaction cycle, showing message submission, Cloud Function invocation, Gemini API call with rolling conversation context, response rendering, and Firestore session update.

4.3 Class Diagram

FinEase's object model is structured in three layers: domain objects (pure data models), service classes (business logic), and repository classes (data access). Table 4.1 summarizes the core domain classes and their key responsibilities. The full UML class diagram is presented in Figure 4.7.

Table 4.1: Core Domain Classes and Responsibilities

Class	Key Attributes	Key Methods / Associations
User	userId, name, Email, language, role, createdAt	Has one Profile; creates many SavingsGoals, LoanSimulations, BudgetPlans, ChatbotSessions, Rewards
SavingsGoal	goalId, userId, title, targetAmount, currentAmount, progressPct, status	recordDeposit() triggers status transition; Completed triggers Reward
Lesson	lessonId, title, language, category, content (Markdown), difficultyLevel	Has one or more Quizzes; fetched by LiteracyService filtered by language
Quiz	quizId, lessonId, questions, passThreshold, pointsAwarded	Evaluated by QuizService; triggers RewardsService on pass
NGOProgram	ngoId, name, category, eligibilityCriteria, isApproved	Has many NGOListings; managed by Admin
LoanSimulation	simulationId, userId, principal, interestRate, tenureMonths, monthlyInstallment	Computed by LoanSimulationService.calculate()
BudgetPlan	planId, userId, income, expenseBreakdown, adviceSummary	Generated by BudgetAdvisorService.generateAdvice()

Class	Key Attributes	Key Methods / Associations
Reward	rewardId (=userId), pointsBalance, badges, dailyPointsEarned, lastEarnedDate	Updated by QuizService and SavingsService; daily cap
ChatbotSession	sessionId, userId, messages, lastUpdatedAt	Belongs to User; managed by ChatbotService; max 10 message pairs

Table 4.1 lists all domain, service, and repository classes in FinEase with their layer, key attributes, primary responsibilities, and the modules each class serves.

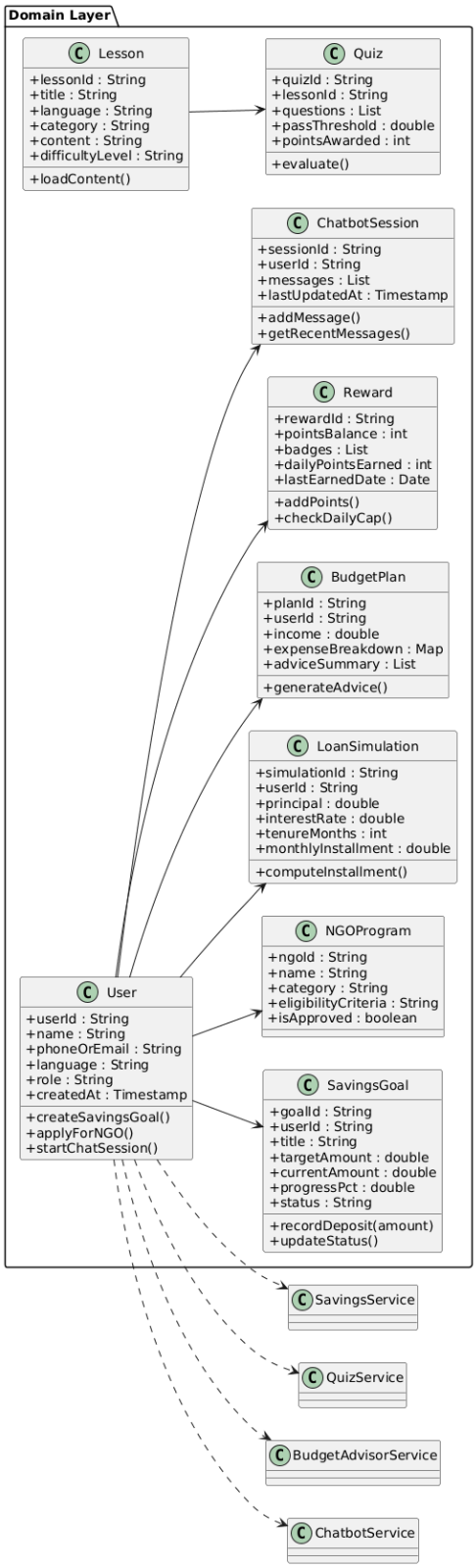

Figure 4.7 a: Class Diagram: FinEase Core Domain and Services

Figure 4.7a illustrates the Domain Layer of the FinEase application, presenting the core entity classes and their relationships. The diagram defines the primary data models including User, Lesson, Quiz, NGOProgram, SavingsGoal, ChatbotSession, Reward, BudgetPlan, and LoanSimulation, each encapsulating their respective attributes and methods. The User class acts as the central entity, maintaining associations with multiple domain objects such as SavingsGoal, BudgetPlan, and ChatbotSession. Supporting service classes including SavingsService, QuizService, BudgetAdvisorService, and ChatbotService are also depicted, outlining the key operations each service is responsible for within the application's business logic layer.

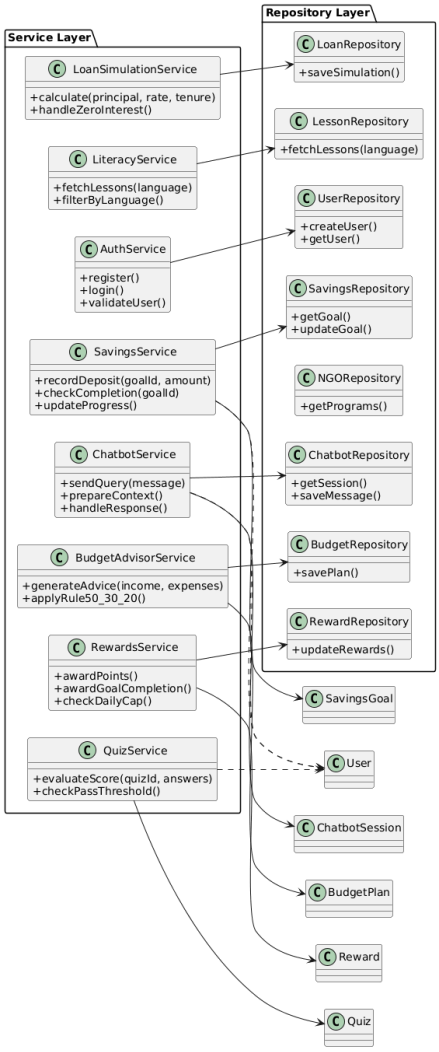

Figure 4.7 b: Class Diagram: FinEase Core Domain and Services

Figure 4.7b presents the complete UML class diagram showing domain objects, service classes, and repository classes with their attributes, methods, and inter-class relationships across all eleven FinEase modules.

This Class Diagram presents the complete FinEase domain model, service classes, and repository classes, showing inheritance relationships, abstract interfaces, associations between entities, and the three layer OOD structure.

4.4 Sequence Diagrams

The following sequence diagrams illustrate the time ordered interactions between actors, Flutter screens, service classes, repository classes, Firebase services, and external APIs for the most critical FinEase workflows.

4.4.1 User Registration and Authentication

The registration sequence (Figure 4.8) involves RegisterScreen captures user input, AuthService.register() creates the account using Firebase Authentication (email and password), FirebaseAuth.createUserWithEmailAndPassword() authenticates the user, UserRepository.createUser() stores the user data in the Firestore users collection, and NavigationService.goToDashboard() completes the process by redirecting the user to the Dashboard.

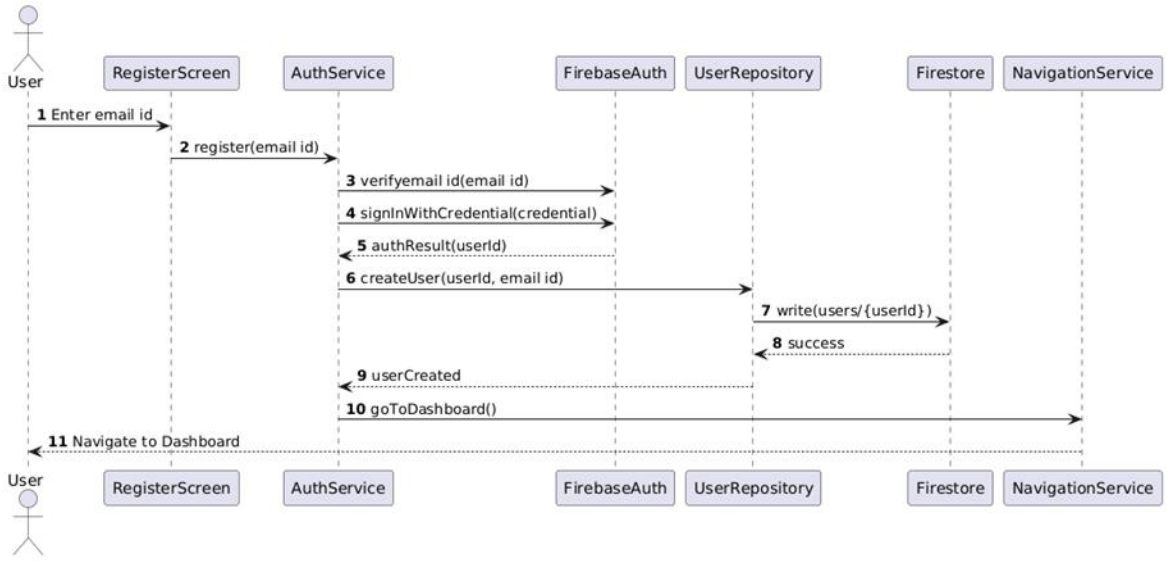

Figure 4.8: Sequence Diagram: User Registration and Authentication

Figure 4.8 shows the message sequence between the Flutter client, Firebase Authentication, and Firestore for email-based registration, including account creation, user authentication, Firestore user document creation, and role-based dashboard routing.

4.4.2 Record Savings Deposit

The deposit sequence Figure 4.9 uses Firestore.runTransaction() for atomic deposit recording: SavingsScreen.onDepositTap() calls SavingsService.recordDeposit(), which runs a Firestore transaction reading current state, computing new values, and writing atomically. On Completed status, RewardsService.awardGoalCompletion() is called. The SavingsScreen updates reactively via Firestore real time stream.

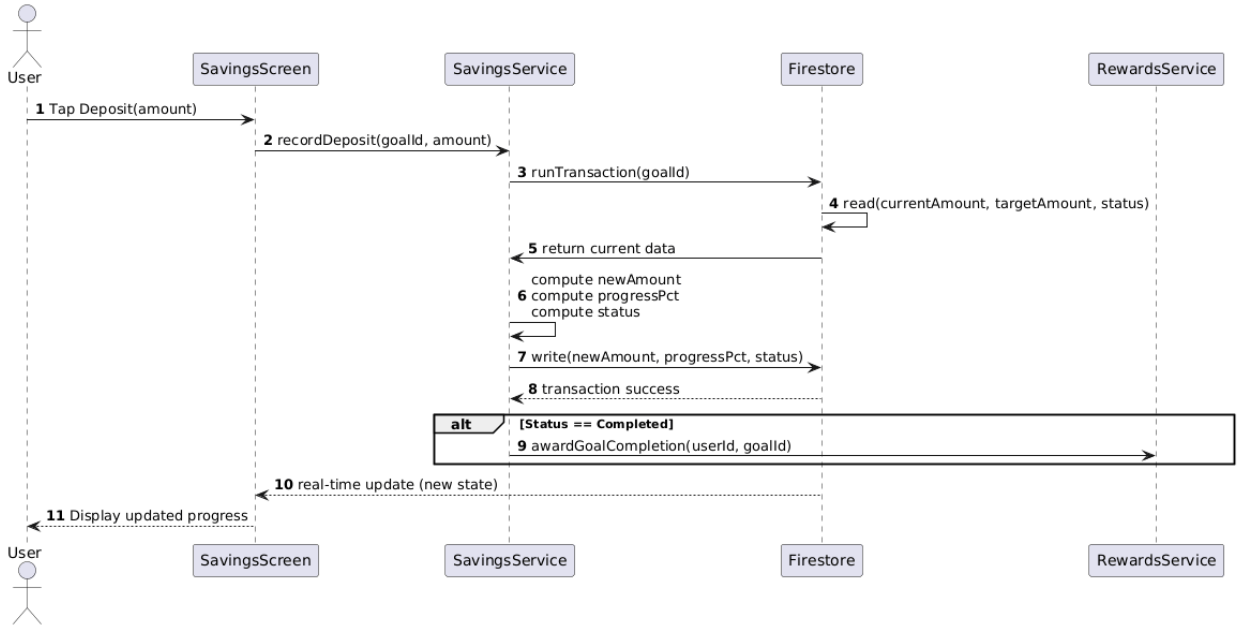

Figure 4.9: Sequence Diagram: Record Savings Deposit

Figure 4.9 illustrates the atomic Firestore transaction flow for recording a savings deposit, including progress calculation, over-deposit capping, goal completion detection, reward issuance, and real-time UI update via stream subscription.

This Sequence Diagram models the deposit recording sequence, showing the user action, SavingsGoalService invocation, Firestore atomic transaction execution, reactive UI update via stream listener, and conditional reward trigger.

4.4.3 AI Chatbot Conversation

The chatbot sequence Figure 4.12 involves: ChatScreen.onSend() calls ChatbotService.sendQuery() which shows a typing indicator and calls a Firebase HTTPS Callable Cloud Function with the message, last 10 session message pairs, language, and user context. The Cloud Function calls the Gemini API with a system prompt, returns the response to the client, and ChatbotRepository.saveMessage() updates the ChatbotSession Firestore document. The chat screen updates reactively. If the Cloud Function is unavailable, a graceful error is shown.

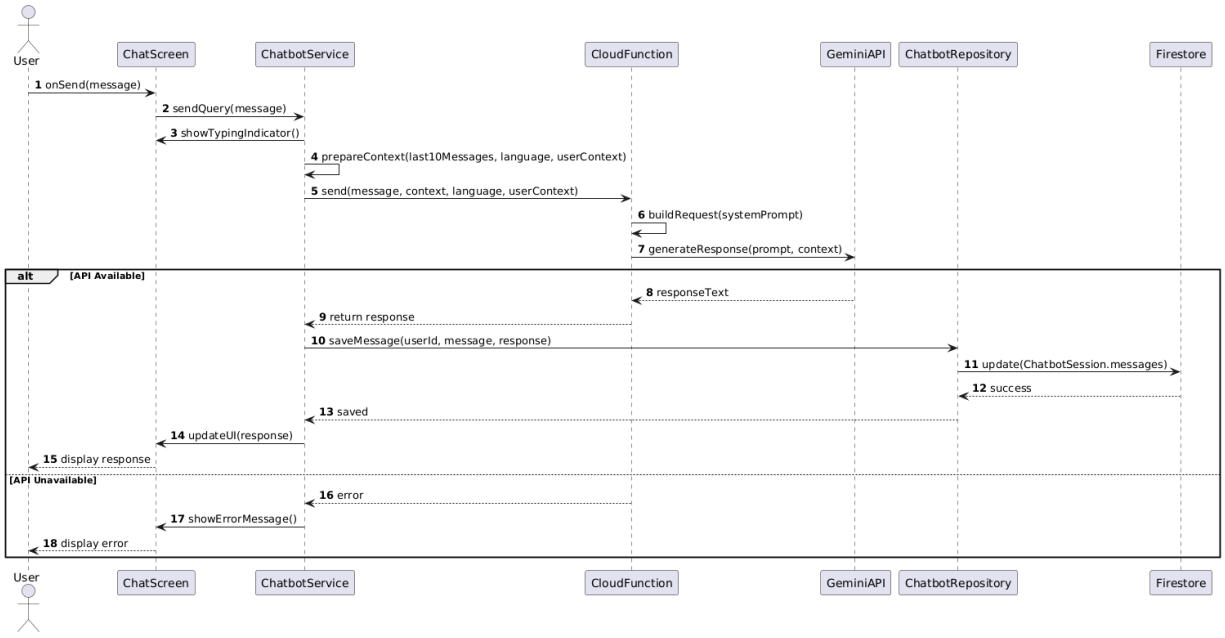

Figure 4.10: Sequence Diagram: AI Chatbot Conversation

Figure 4.10 depicts the multi-component message sequence for a chatbot query: ChatScreen to ChatbotService to Cloud Function to Gemini API, showing context passing, response return, and session history update in Firestore. This Sequence Diagram traces the chatbot conversation turn, showing ChatScreen message submission, ChatbotService Cloud Function call with conversation history context, Gemini API response, session document update, and UI rendering.

4.5 State Transition Diagrams

4.5.1 AI Budget Advisor

The AI Budget Advisor state machine has six states: Idle (user opens dashboard) → Step 1 (enter monthly income in PKR) → Step 2 (enter categorized expenses) → Rule Engine (apply 50/30/20

rule and calculate deviations) → Rank & Advice (rank issues and generate up to 5 simple suggestions) → Display Advice (show advice in plain language) → Persist (save budget to Firestore). Connectivity guard: if online, Gemini API enhances advice; if offline or failed, local advice is used. Step 2 allows back navigation to Step 1 and retry on invalid input. Persist handles save failures by notifying the user but still allowing access to history.

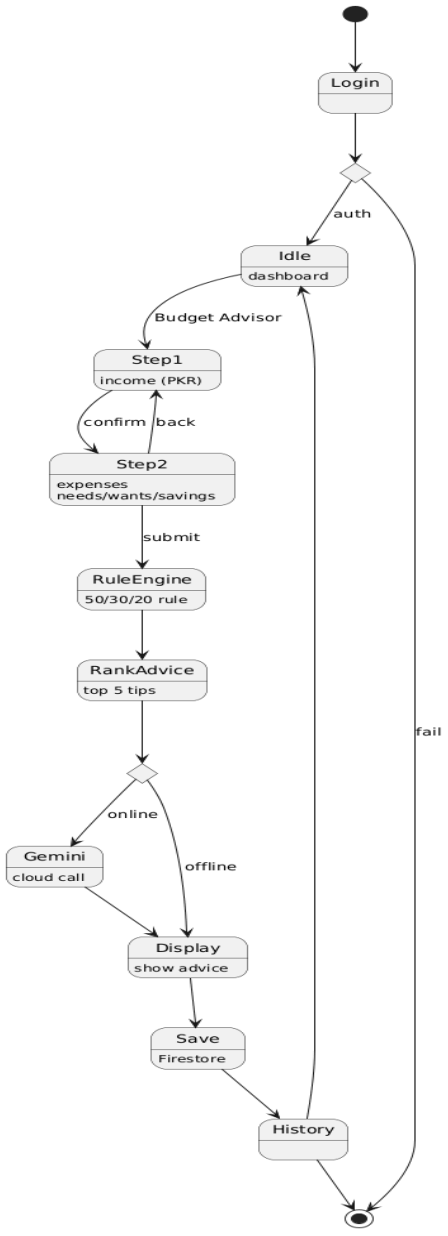

Figure 4.11: State Transition Diagram: AI Budget Advisor

Figure 4.11 shows an 8-state budget advisory flow: Idle → Step 1 (Income) → Step 2 (Expenses) → Rule Engine (50/30/20 analysis) → Rank & Advice → Gemini Invoke (if online) → Display Advice → Persist (Firestore save). It runs in the BudgetAdvisorService of FinEase: UI handles steps, service layer runs logic, Firebase handles storage, and Cloud Functions optionally provide Gemini-enhanced advice.

4.5.2 Savings Goal Lifecycle

The Savings Goal state machine, as shown in Figure 4.14, models four states: Created (goal written, FCM reminder scheduled) to Active (first deposit recorded) to Completed (target reached, reward issued) or Cancelled (user confirmed). Deposit recording uses Firestore transactions.

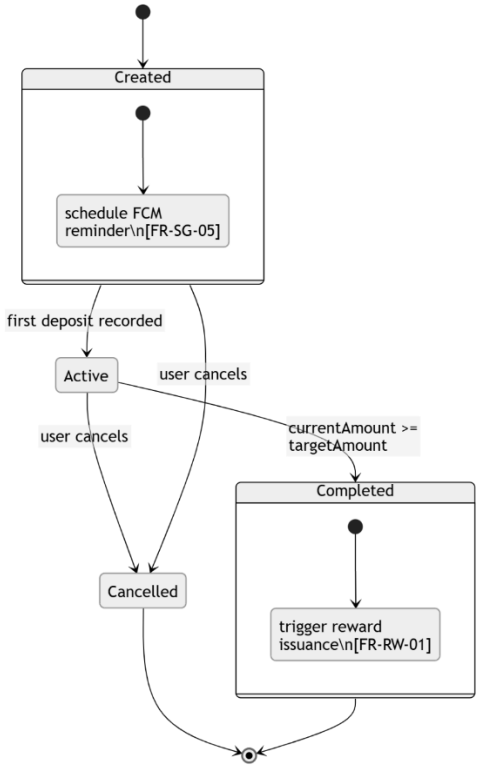

Figure 4.12: State Transition Diagram: Savings Goal Lifecycle

Figure 4.12 illustrates the savings goal state machine with transitions between Created, Active, Completed, and Cancelled states, showing deposit-driven progress updates, automatic completion at 100%, and the conditions that trigger each transition.

This State Transition Diagram models the Savings Goal lifecycle, showing transitions between Created, Active, Completed, and Cancelled states, with conditions for each transition including deposit thresholds, target date proximity, and user cancellation actions.

4.6 Data Design

FinEase uses Firebase Cloud Firestore as its primary data store, organised in fourteen top level collections aligned to the domain model, as shown in Figure 4.7. The data design follows four governing principles: each domain entity maps to a dedicated top level collection; the Firebase Authentication userId serves as the universal foreign key linking all user owned documents; frequently co accessed data is denormalised by embedding it within parent documents to minimise read counts per screen load; and infrequently changing reference data (lesson content, NGO listings) is cached client side via Firestore offline persistence . Firebase Security Rules enforce per user data isolation and Admin Custom Claims at the database level, independently of client application behavior.

4.7 Data Representation

This section presents the main data representations used throughout FinEase, illustrating how information flows between the user interface, application services, and the Firebase backend. Each diagram and description below captures a distinct aspect of data structure, lifecycle, or flow within the system. Together they provide a comprehensive picture of how FinEase stores, retrieves, transforms, and presents data across its eleven modules.

4.7.1 Firestore Document Hierarchy

The Firestore document hierarchy illustrates how FinEase's fourteen top level collections are structured and how documents within each collection relate to user identities through the userId foreign key. The hierarchy begins at the Firebase project root, branches into top level collections (users, savings_goals, lessons, quizzes, ngo_programs, ngo_applications, loan_simulations, budget_plans, rewards, chatbot_sessions, forum_threads, forum_posts, rewards, partner_listings), and shows how sub collections are used where nesting improves query efficiency. This diagram is essential for understanding data isolation (each user's documents are owned by their userId), access

control (Security Rules reference userId fields), and offline persistence boundaries (collections vs. individual documents).

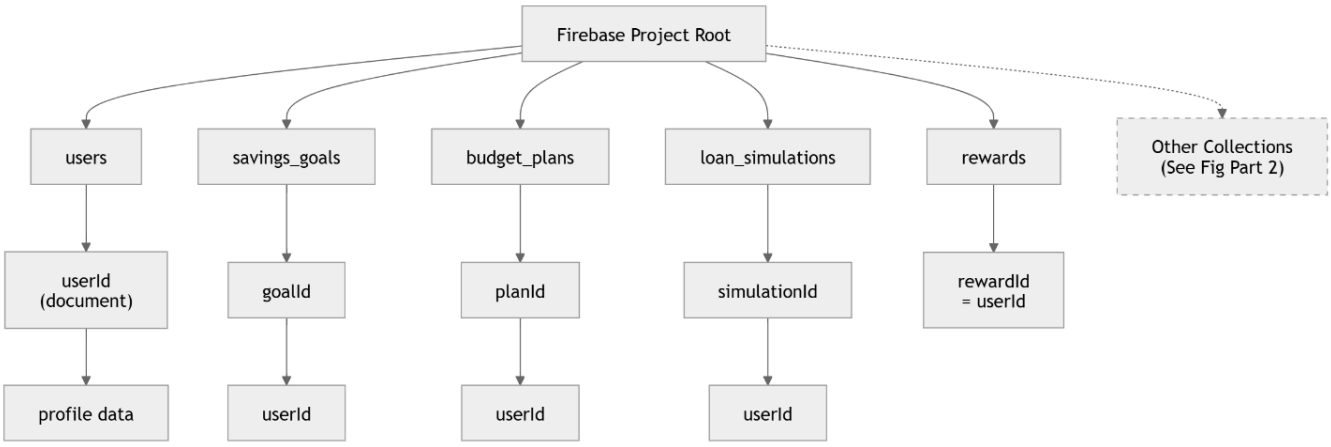

Figure 4.13 a Firestore Document Hierarchy for FinEase

Figure 4.13a illustrates the first part of the Firestore document hierarchy for the FinEase application, rooted at the Firebase Project Root. It displays five primary top-level collections: users, savings_goals, budget_plans, loan_simulations, and rewards, along with a reference to other collections covered in Part 2. Each collection is structured with a unique document identifier such as userId, goalId, planId, simulationId, and rewardId-userId respectively ensuring proper data ownership and isolation per user.

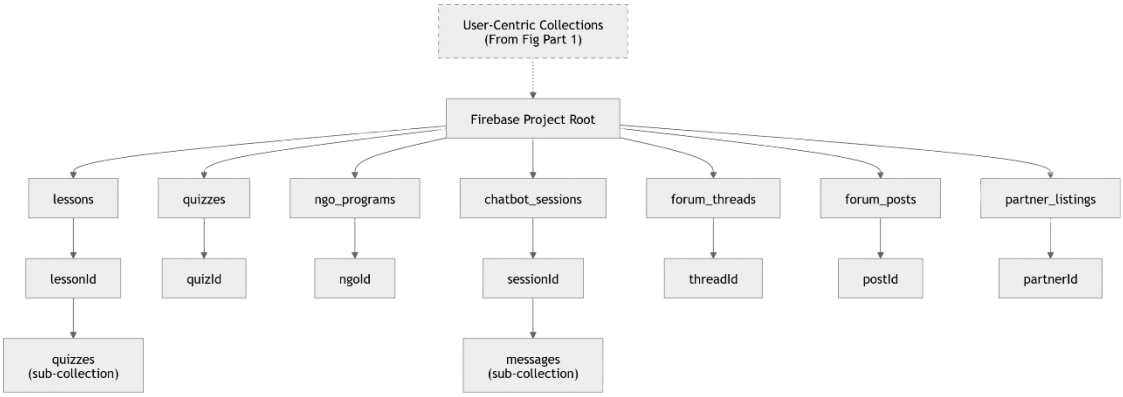

Figure 4.13 b Firestore Document Hierarchy for FinEase

Figure 4.13b displays the complete Firestore collection hierarchy, showing the fourteen top-level collections, their document key structures, and the subcollection relationships used to organize all FinEase data. This diagram illustrates the Firestore document hierarchy for FinEase, showing all

fourteen top level collections rooted at the Firebase project, sample document paths with userId keyed ownership, and key embedded sub fields such as the messages array in chatbot sessions and questions array in quiz documents.

4.7.2 User Profile Data Model

The User Profile data model captures all fields stored in the users Firestore collection for each user. The primary key is userId (Firebase Authentication UID), which links all user owned documents across collections. Core profile fields include: name (String), Email (String), language preference (String: "ur" or "en"), role (String: "user" or "admin"), estimated monthly income in PKR (Number), risk tolerance level (String: "low", "medium", "high"), createdAt timestamp, and lastLoginAt timestamp. The language field drives the entire bilingual experience, while the income field feeds the loan affordability indicator and budget advice generation. The role field, cross validated against Firebase Custom Claims, determines dashboard routing on authentication.

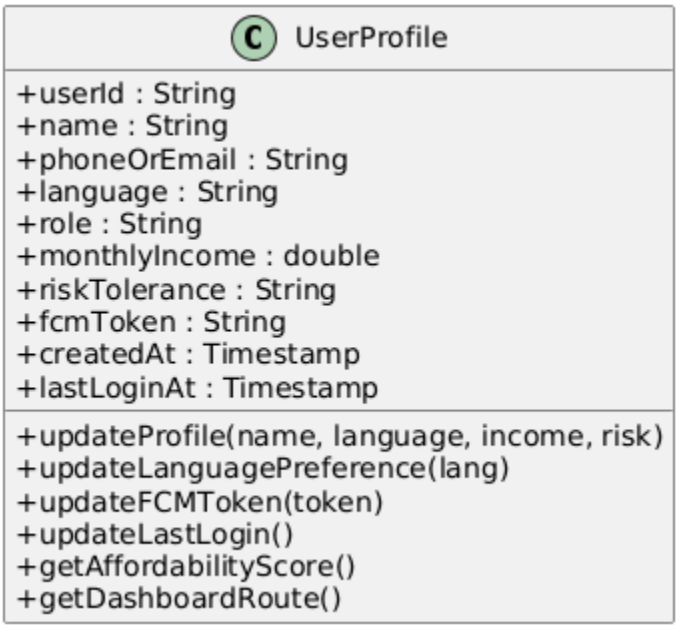

Figure 4.14 User Profile Data Model and Collection Relationships

Figure 4.14 shows the User Profile Firestore document schema including all fields, data types, and the foreign key relationships linking the user document to savings goals, chatbot sessions, NGO applications, budget plans, and rewards records. This entity diagram presents the User Profile document schema, showing all fields with their data types and constraints, and relationship arrows

indicating how the userId field links to the savings_goals, ngo_listings, rewards, chatbot_sessions, and budget_plans collections.

4.7.3 Savings Goal Lifecycle Data

The Savings Goal data model captures the state and progress information for each user created savings goal. Key fields include: goalId (String, Firestore auto generated), userId (String, foreign key), title (String), targetAmount (Number, PKR), currentAmount (Number, PKR, updated atomically), progressPercentage (Number, 0 100, derived), status (String: Created/Active/Completed/Cancelled), targetDate (Timestamp), createdAt (Timestamp), and completedAt (Timestamp, nullable). The currentAmount and progressPercentage fields are always updated together within a single Firestore transaction to guarantee consistency. The status field drives the UI presentation and controls whether new deposits are accepted. This data model directly maps to the Savings Goal Lifecycle state machine illustrated in Figure 4.14.

Figure 4.15 Savings Goal Collection Schema and Transaction Boundary

Figure 4.15 details the SavingsGoal Firestore document schema, the DepositRecord subcollection structure, and the transaction boundary that atomically updates currentAmount, progressPercentage, and status fields during a deposit operation.

This diagram presents the Savings Goal collection schema with all field definitions, the status enum lifecycle, and annotated boundaries around the currentAmount, progressPercentage, and status fields that are always updated together within a single Firestore atomic transaction.

4.7.4 Chatbot Session Message Array

The Chatbot Session data model stores the conversation history between a user and the Gemini powered AI chatbot. Each chatbot_sessions document is keyed by a combination of userId and sessionId and contains a messages array field holding up to ten message pair objects. Each message object in the array contains: role (String: "user" or "model"), content (String, the message text in the user's chosen language), and timestamp (Timestamp). The array is bound to ten pairs to manage Firestore document size limits and to control the context window sent to the Gemini API on each request. The lastUpdatedAt field enables chronological ordering of sessions in the conversation history view. The entire messages array is transmitted to the Cloud Function on each new message, providing the Gemini model with sufficient context for coherent multi turn conversation. This bounded FIFO structure is a practical application of queue based data management discussed in the Data Structures course.

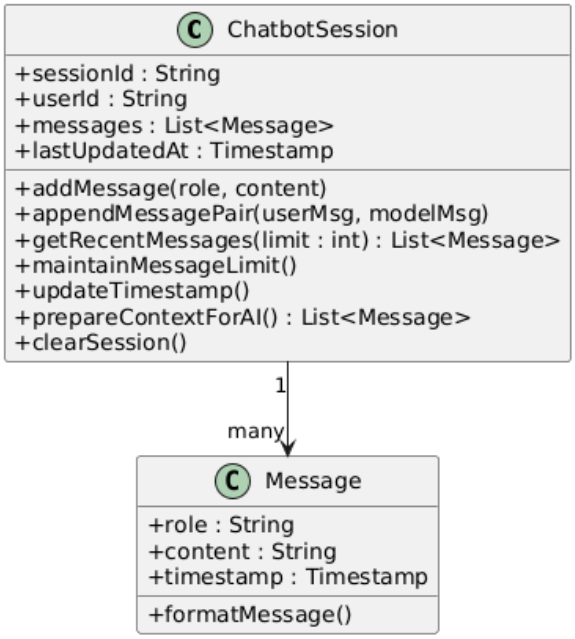

Figure 4.16 Chatbot Session Schema and Message Flow

Figure 4.16 presents the Chatbot Session document schema showing the messages array structure, message pair format (user query and AI response), language field, and the rolling 10-pair window used as conversation context for the Gemini API. This diagram shows the chatbot session document schema, the messages array structure with role, content, and timestamp fields per element, the ten pair FIFO boundary, and the complete data flow from ChatScreen through ChatbotService, Cloud Function, Gemini API, and back to Firestore.

4.7.5 NGO Listing State Data

The NGO Listing data model captures the full lifecycle of a user's welfare program application from initial draft through final disposition. Key fields include: userId (String, applicant foreign key), ngoId (String, program foreign key), reviewNotes (String, admin provided, visible to applicant), submittedAt (Timestamp), reviewedAt (Timestamp, nullable), and updatedAt (Timestamp).

Figure 4.17 NGO Listing Schema and Status Transition Data

Figure 4.17 specifies the NGO Listing Firestore document schema including status field values, reviewNotes array and the timestamp fields that record each state transition for audit purposes.

This diagram presents the NGO Application document schema, the status enum lifecycle aligned with the state transition diagram, the Firebase Security Rules access boundaries distinguishing user writable and admin only status transitions.

Table 4.2: Data Dictionary: Key Firestore Collections and Attributes (Selected)

Collection	Field	Type	Description
users	userId	String	Firebase Auth UID (primary key)
users	language	String	ur for Urdu, en for English
users	role	String	user or admin (Custom Claims for admin)
savings_goals	targetAmount	Number	Target savings amount in PKR
savings_goals	currentAmount	Number	Amount saved so far (updated by transaction)
savings_goals	progressPercentage	Number	(currentAmount / targetAmount) x 100
savings_goals	status	String	Created / Active / Completed / Cancelled Figure 4.14
lessons	language	String	ur or en
lessons	content	String	Full lesson content as Markdown
lessons	difficultyLevel	String	Beginner / Intermediate / Advanced
quizzes	passThreshold	Number	Minimum score to pass (default: 80%)
quizzes	pointsAwarded	Number	Points awarded on passing (default: 10)
rewards	pointsBalance	Number	Current total points balance
rewards	dailyPointsEarned	Number	Points earned today (reset daily)
chatbot_sessions	messages	Array	Array of {role, content, timestamp} max 10 pairs
budget_plans	adviceSummary	Array	Up to 5 generated advice strings

Table 4.2 enumerates all fourteen Firestore collections with their document key structure, primary fields, data types, cardinality, ownership rules, and the modules that read and write each collection.

Chapter 5

Implementation

5 Implementation

The implementation of FinEase translates the approved requirements and design models into a functioning software product. It combines Flutter frontend development, Firebase backend configuration, AI integration and Firestore data connectivity into one coordinated system. The implementation follows the multi tier architecture described in the SDD and reflects the Agile incremental strategy selected during project planning. Each increment focused on a specific set of modules that could be developed, tested, and demonstrated as a working vertical slice before the next increment began.

5.1 Development Environment and Project Structure

The FinEase Flutter project follows a feature first folder structure, as shown in Figure 5.1, grouping files by feature module rather than by file type. This approach improves code discoverability, makes module boundaries explicit, and maintains the separation between modules central to OOD. Each feature folder contains its own screens, widgets, and feature specific utilities, while shared code (models, repositories, services) resides in the top level data/ directory.

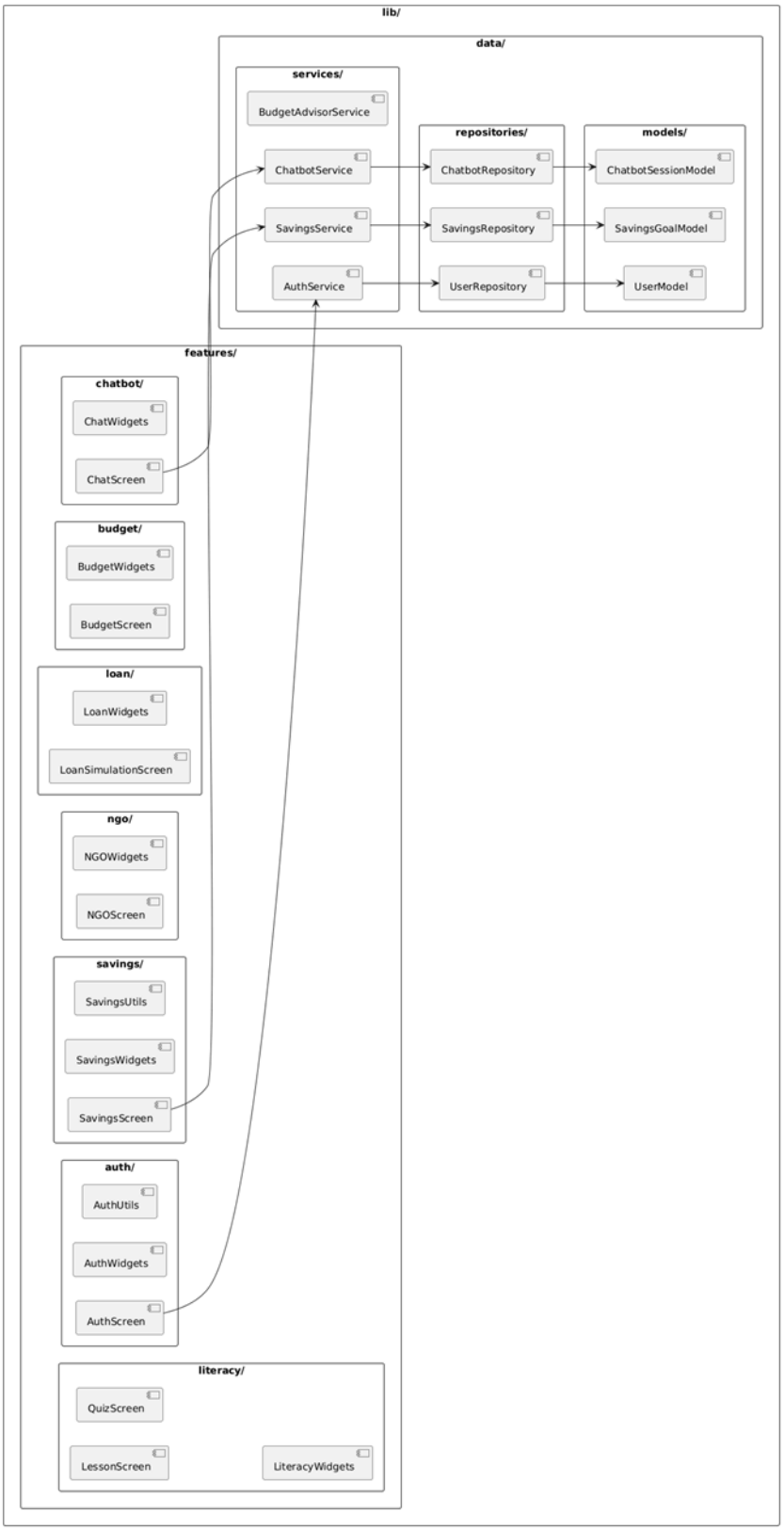

Figure 5.1: Flutter Project Structure: Feature First Architecture

Figure 5.1 shows the feature-first directory layout of the Flutter project, including the lib/features/ module folders, shared/ utilities, and the separation of screens, services, and repositories within each feature directory.

Firebase was initialised using the FlutterFire CLI (flutterfire configure), which generated the firebase_options.dart file. Firebase services configured include Authentication, Cloud Firestore (production mode, offline persistence enabled in main.dart), Cloud Functions (Node.js 18, us central1 region), Cloud Messaging (Android), and Storage. Firestore Security Rules were deployed via Firebase CLI. The Firebase architecture is shown in Figure 5.2.

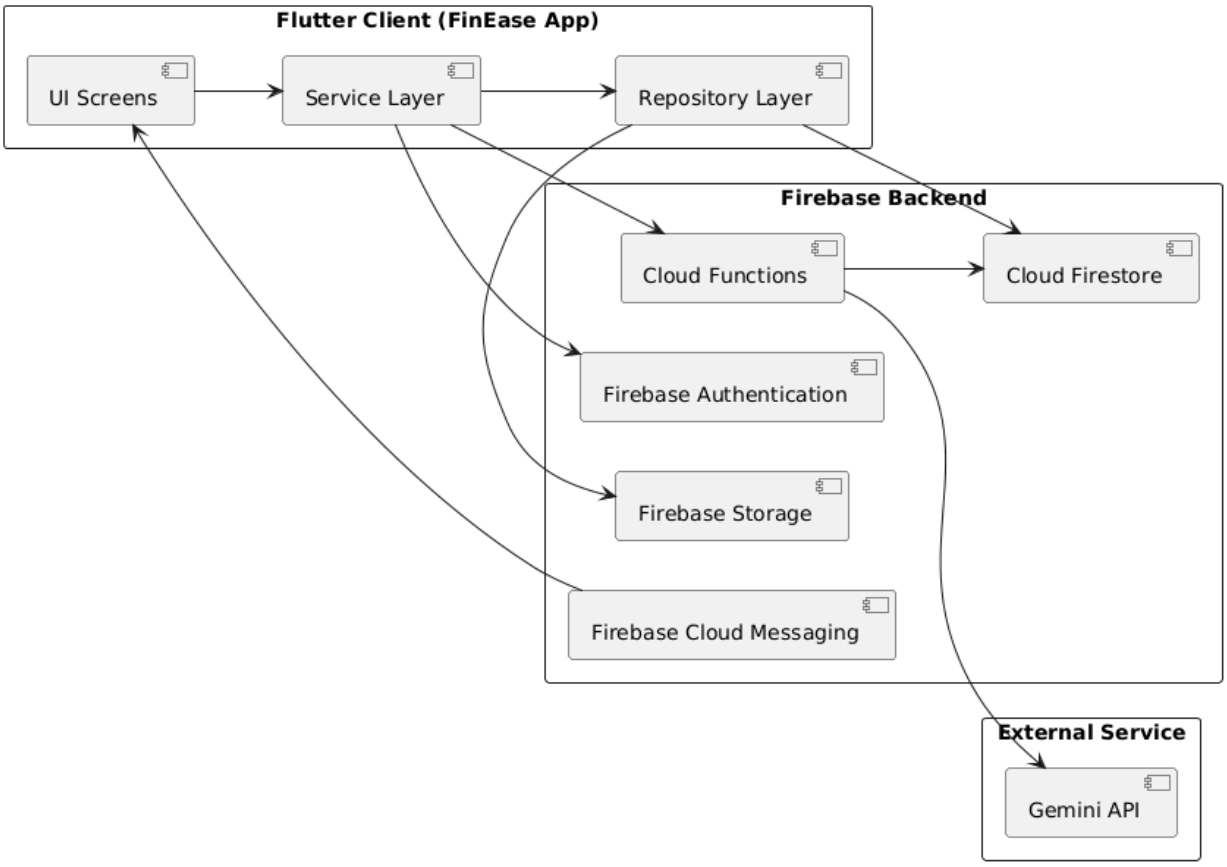

Figure 5.2: Firebase Architecture and Services Used

Figure 5.2 illustrates how the FinEase Flutter client connects to Firebase Authentication, Cloud Firestore, Cloud Functions, Firebase Cloud Messaging, and Firebase Storage, showing the service boundaries and data flow between each component.

This diagram illustrates the Firebase cloud architecture used by FinEase, showing the Firebase Authentication, Cloud Firestore, Cloud Functions, Cloud Messaging, and Storage services, their inter service interactions, and the Android client SDK connections.

Table 5.1: Flutter Package Dependencies

Package	Version	Purpose
firebase_core	^2.24.0	Firebase app initialization
firebase_auth	^4.15.0	Email OTP authentication
cloud_firestore	^4.13.0	All Firestore database operations including offline persistence Table 4.2
cloud_functions	^4.5.0	HTTPS Callable Cloud Functions for Chatbot and Budget Advisor
provider	^6.1.0	Dependency injection and reactive state management
fl_chart	^0.65.0	Charts: savings progress bar, budget pie chart, loan repayment chart
flutter_markdown	^0.6.0	Rendering Markdown formatted lesson content
intl	^0.18.0	Date/time formatting and PKR currency formatting
connectivity_plus	^5.0.0	Detecting network status for graceful degradation
lottie	^2.7.0	Animated illustrations for onboarding and goal completion

Table 5.1 lists all Flutter/Dart package dependencies with version constraints, primary purpose, and the module(s) that utilise each package.

5.2 Authentication Implementation

Authentication is handled by AuthService wrapping Firebase Authentication. Firebase Authentication UIDs serve as primary keys across all users owned Firestore documents, so that

each user can only access their own data: allow read, write: if request.auth.uid == resource.data.userId. Role-based access control is implemented via Firebase Custom Claims set using the Firebase Admin SDK.

5.3 Key Module Implementations

5.3.1 Financial Literacy Hub

The hub works as a three screen flow: LessonBrowserScreen (Firestore real time listener on lessons collection filtered by language) to LessonScreen (Markdown rendering with RTL support via flutter_markdown) to QuizScreen (PageView with progress indicator). Quiz evaluation runs client side. On pass, a Firestore transaction atomically awards points and enforces the daily cap. Badge conditions are evaluated post transaction. Offline access is automatic via Firestore persistence.

5.3.2 Savings Goal Tracker

The Savings module works when the SavingsGoalListScreen subscribes to a Firestore real time query filtered by userId, showing up-to-date progress data without needing to refresh. Deposit recording uses Firestore.runTransaction() as specified in Section 5.9.2.

5.3.3 NGO Directory

The NGO Directory displays the NGO listings by loading data from Firestore. The system shall maintain a directory of NGO programs with name, category, eligibility criteria, and application steps. Users shall be able to search programs by name keyword. The user will then click on the desired NGO listing. It will display information about the listing along with programs and details. The navigation button will lead user to the official website of NGO.

5.3.4 AI Chatbot

Chatbot functions across four systems: ChatScreen UI, ChatbotService (Dart), Cloud Function (Node.js), and Gemini API. ChatbotService passes the last 10 message pairs with each query for contextual responses. The Gemini API key is stored as a Firebase secret using firebase_functions: secrets: set GEMINI_API_KEY never in the client APK or repository. Quick action suggestion chips reduce the cold start barrier for first time users.

5.4 Algorithms

5.4.1 Loan Amortization Formula

Input: principal P (PKR), annualInterestRate r_annual (%), tenureMonths n. Algorithm: r_monthly = r_annual / 12 / 100. If r_monthly = 0: M = P / n (zero rate edge case). Else: M = P x [r_monthly x (1+r_monthly)^n] / [(1+r_monthly)^n 1] (standard amortization). T = M x n. I = T P. Output: M (monthly installment), T (total repayable), I (total interest) all in PKR. Complexity: O(1).

5.4.2 Savings Goal Progress Update

Input: goalId (String), depositAmount d (PKR). Algorithm (inside Firestore Transaction): read goalDoc. Validate d > 0. newCurrent = min(goalDoc.currentAmount + d, goalDoc.targetAmount). progress = (newCurrent / goalDoc.targetAmount) x 100. newStatus = if newCurrent >= targetAmount then Completed else Active. Atomically write updated document. If Completed: call RewardsService.awardGoalCompletion(). Complexity: O(1).

5.4.3 Rule Based Budget Advice

Input: income Y (PKR), expensesByCategory E. Algorithm: needsCap = 0.50 x Y; wantsCap = 0.30 x Y; savingsCap = 0.20 x Y. Compute totalNeeds and totalWants from categorised expenses. Generate advice items: if totalNeeds > needsCap: "Reduce essential expenses by PKR X"; if totalWants > wantsCap: "Reduce discretionary spending by PKR X"; if impliedSavings < savingsCap: "Save at least PKR X more per month". Add category specific items sorted by overspend magnitude, capped at 5 total . Complexity: O(n log n) for category sorting.

5.5 External APIs

Table 5.2: External APIs Used in FinEase

API	Purpose in FinEase	Security Model
Firebase Auth (signInWithCredential)	User registration and login via OTP	OTP expires 5 min; JWT session tokens; HTTPS
Cloud Firestore (collection, runTransaction, onSnapshot)	All persistent data storage across 14 collections Table 4.2	Security Rules; Custom Claims for admin; HTTPS

API	Purpose in FinEase	Security Model
Cloud Functions httpsCallable(chatbot, getBudgetAdvice)	Relay messages to Gemini API; enhanced budget advice	HTTPS callable requires authenticated session; Gemini key in secrets
Firebase Cloud Messaging (Admin SDK send)	Savings reminders and NGO status updates	Server side send only via Cloud Functions; FCM tokens in Firestore
Gemini API (generateContent)	AI Chatbot and enhanced budget advice	Key in Cloud Function secret only; never in client APK

Table 5.2 catalogues all external APIs and Firebase services integrated in FinEase, specifying the SDK or endpoint used, authentication mechanism, and the module(s) each service supports.

5.6 User Interface Design

FinEase's user interface is built using Flutter's Material Design 3 component library, adapted for accessibility, bilingual operation, and the specific usability needs of financially underserved users. The app uses Teal (#009688) as the primary brand color and Green (#4CAF50) as the secondary accent, to give a feeling of trust and growth. Typography uses Nunito (minimum 14sp body, 18sp headings) for Latin script and Noto Nastaliq Urdu for Urdu script, with all contrast ratios maintaining a at least 4.5:1. All interactive targets are sized at a minimum of 48x48 dp. All main tasks can be reached in three taps from the main dashboard. The following subsections describe each major screen of the application with corresponding screenshots.

5.6.1 Onboarding and Registration Screen

The Registration Screen is the entry point for new users and is designed to present a welcoming, welcoming first experience. The screen displays the FinEase logo with a brief value proposition in the user's device language. Users register with their email. UI also asks for his/her full name and password. Inline validation provides immediate feedback on invalid formats. The registration flow concludes with a biometric enable or disable toggle button. Error states (duplicate accounts) are presented with actionable guidance rather than technical error codes, reflecting HCI principles of error recovery support.

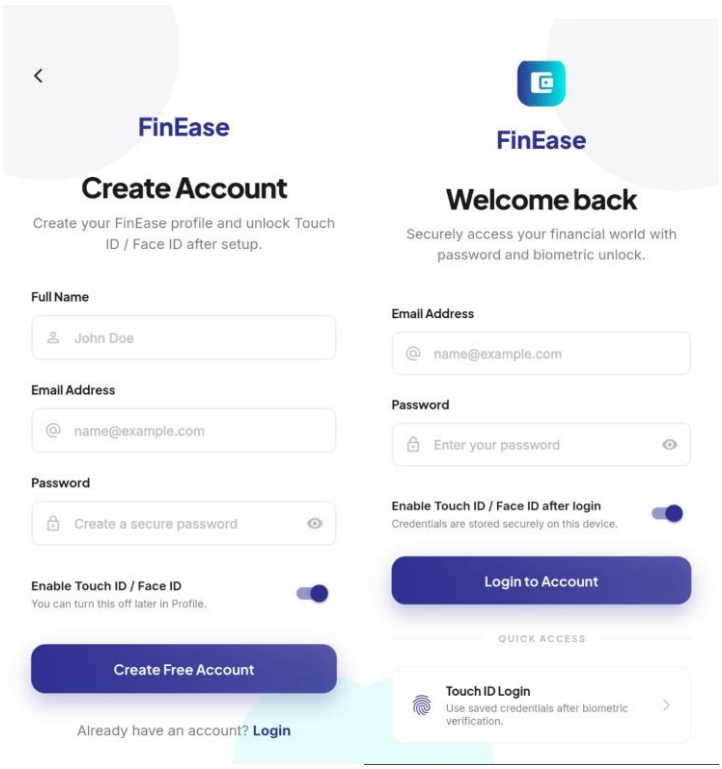

Figure 5.3 User Registration Screen

These screenshots display profile setup form where the user sets name, email and password for his account. These also show login page for signing in. The screen uses Teal branding, minimal cognitive load design, and inline validation to guide first time users through the two step authentication flow.

5.6.2 Main Dashboard

The Main Dashboard is the central hub of FinEase and is designed to surface the most actionable information at a glance while providing navigation to all eleven modules within three taps. The dashboard displays a personalized greeting using the user's name, a summary card showing active savings goals with progress bars, a featured lesson card from the Financial Literacy Hub, and quick action tiles for the AI Chatbot and Budget Advisor. A bottom navigation bar provides direct access to the five primary tab sections: Dashboard, Learn, Save, Support, and Rewards. The dashboard subscribes to real time Firestore listeners so that savings goal progress and reward points update reactively without requiring manual refresh.

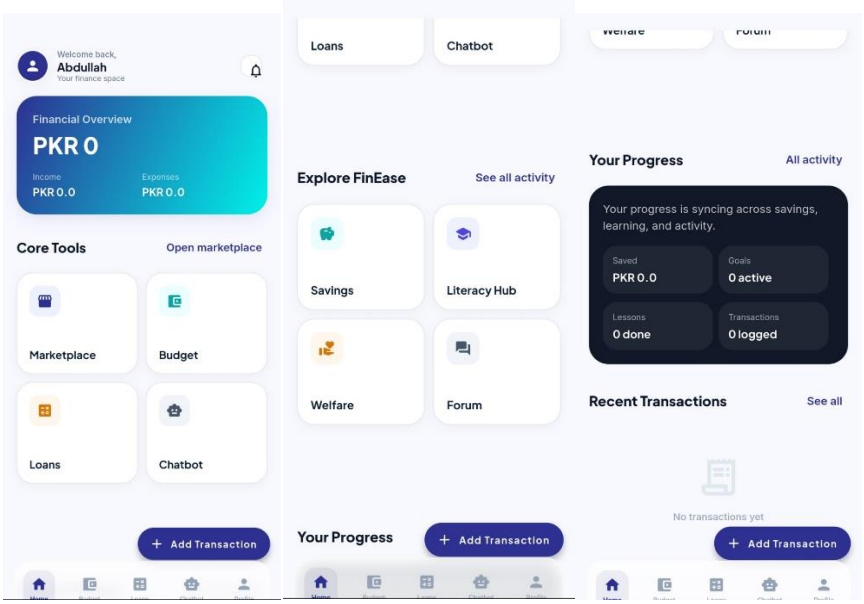

Figure 5.4 Main Dashboard Screen

This screenshot shows the personalized main dashboard presenting the user's name, active savings goal summary, featured lesson card, reward points balance, and quick-access navigation tiles for all eleven FinEase modules. The Main Dashboard shown above displays the personalized greeting, active savings goal summary card with a linear progress bar, a featured lesson recommendation, quick action tiles for the AI Chatbot and Budget Advisor, and the bottom navigation bar providing access to all five primary sections.

5.6.3 Financial Literacy Hub Screen

The Financial Literacy Hub screen presents lesson content organized by topic category (Savings, Budgeting, Loans, Investment Basics, Welfare Programs) and difficulty level (Beginner, Intermediate, Advanced) using a horizontally scrollable chip filter row and vertically scrollable lesson cards. Each lesson card displays the topic title, difficulty badge, estimated reading time, and completion status indicator (not started, in progress, completed). Tapping a lesson opens the Lesson Screen, which renders Markdown formatted content using the flutter_markdown package with correct RTL text direction for Urdu lessons. A Continue to Quiz button is anchored at the bottom of the lesson content.

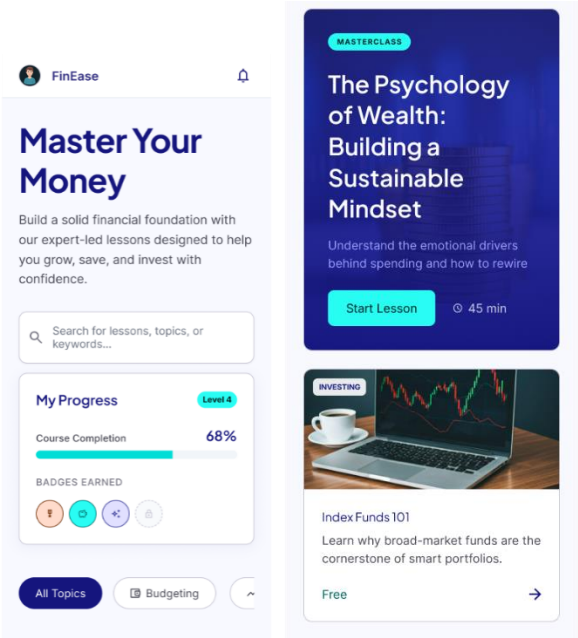

Figure 5.5 Financial Literacy Hub: Lesson Browser, Lesson Screen, and Quiz Screen

These screenshots present three screens: the categorized lesson browser with filter chips, a lesson detail screen with rich text content, and the post-lesson quiz screen showing a multiple-choice question with the score and points award on completion. The Financial Literacy Hub screens shown above illustrate the lesson browser with category filter chips, a rendered Markdown lesson page, and a quiz question screen with multiple choice options and a progress indicator.

5.6.4 Savings Goal Tracker Screen

The Savings Goal Tracker screen displays the user's active and historical savings goals as a vertically scrollable list of goal cards. Each card shows the goal title, target amount, current amount, a linear progress bar showing percentage completion, and the target date with a days remaining counter. A floating action button opens the goal creation modal, which collects the goal title, target amount (with PKR currency formatting), and target date using a date picker widget. The deposit recording flow is accessible via a Record Deposit button on each goal card, presenting a bottom sheet with an amount input field and a confirmation button.

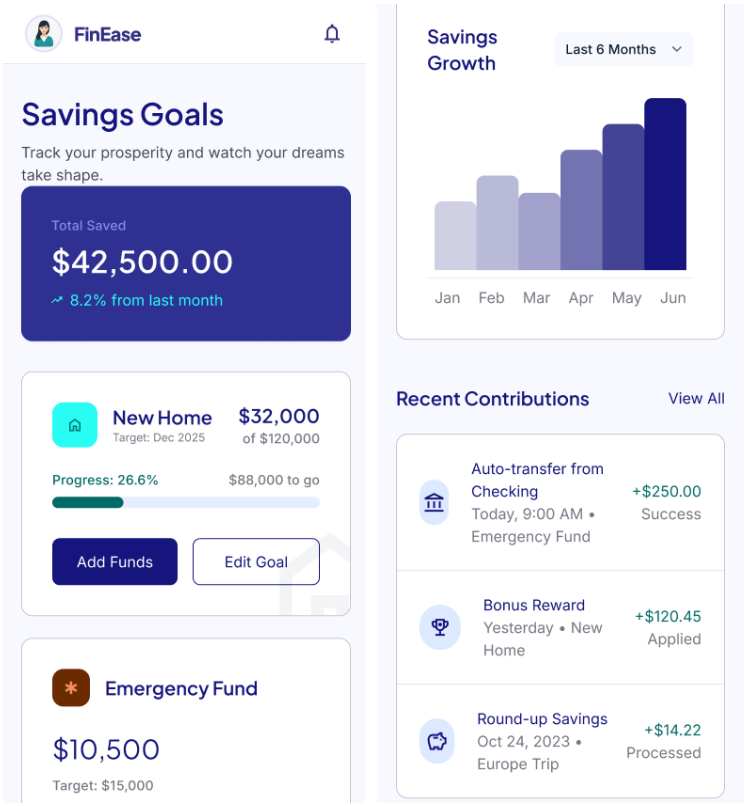

Figure 5.6 Savings Goal Tracker: Goal List, Creation Form, and Deposit Screen

These screenshots show the savings goal list with circular progress indicators, the goal creation form with title, target amount, and deadline fields, and the deposit recording screen displaying the updated progress bar and current balance after a successful deposit. The Savings Goal Tracker screens shown above present the active goals list with progress bars and days remaining counters, the goal creation form with title, target amount, and date picker and the deposit recording bottom sheet with confirmation flow.

5.6.5 NGO Directory

The NGO Directory screen provides a searchable, filterable list of approved welfare programs sourced from Firestore. A search bar at the top accepts keyword input with a 300 millisecond debounce for responsive filtering. Category filter chips (Financial Aid, Health, Education, Housing, Food Security) below the search bar allows single category filtering. Each program card displays the name, category badge, a brief eligibility summary, and an Apply button. Tapping a card opens the Program Detail Screen showing full eligibility criteria and step by step application guidance.

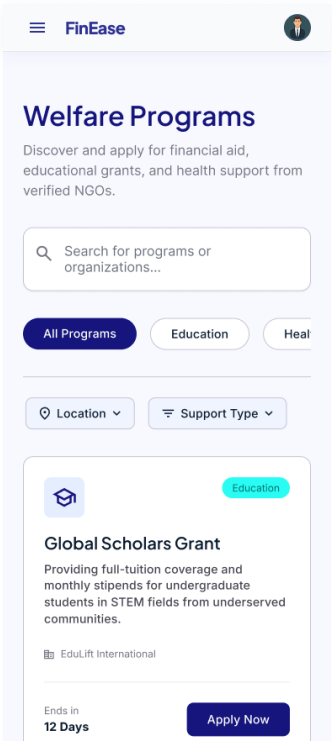

Figure 5.7 NGO Directory: Program Listing, Detail, and Status View

This screenshot displays four screens: the searchable NGO program list with category filter chips, a program detail card with eligibility criteria and contact information. The NGO Directory screens shown above display the searchable program listing with category filter chips and a program detail page with eligibility criteria.

5.6.6 AI Chatbot Screen

The AI Chatbot screen provides a simple chat interface with a message history, input field, and send button. User messages appear on the right in teal bubbles, while AI responses appear on the left in light grey bubbles. Urdu messages are displayed in right-to-left format using a suitable Urdu font. If there are no previous messages, the screen shows suggested questions to help users get started (e.g., saving money or learning about financial programs). When a message is sent, a typing indicator (three animated dots) is shown until the AI response is received.

.

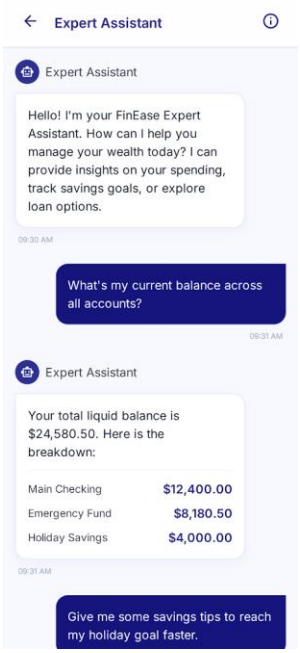

Figure 5.8 AI Chatbot Screen: Initial State, English Conversation, and Urdu Conversation

This screenshot shows the chatbot in three states: the initial screen with suggested questions, an English conversation with financial advice, and an Urdu conversation with right-to-left text. The chatbot displays user messages in teal bubbles and AI responses in grey bubbles. It also supports both English and Urdu, with proper formatting for each language.

5.6.7 Loan Simulation Screen

The Loan Simulation screen provides a straightforward three input form for principal amount (PKR), annual interest rate (%), and repayment tenure (months), each with inline validation preventing non numeric input and out of range values. A Calculate button triggers the client side amortization computation. Results are displayed in a summary card showing monthly installment, total repayable amount, and total interest cost, all formatted in PKR with thousands of separators for readability. Educational contextual text below the results panel explains the amortization formula in plain language.

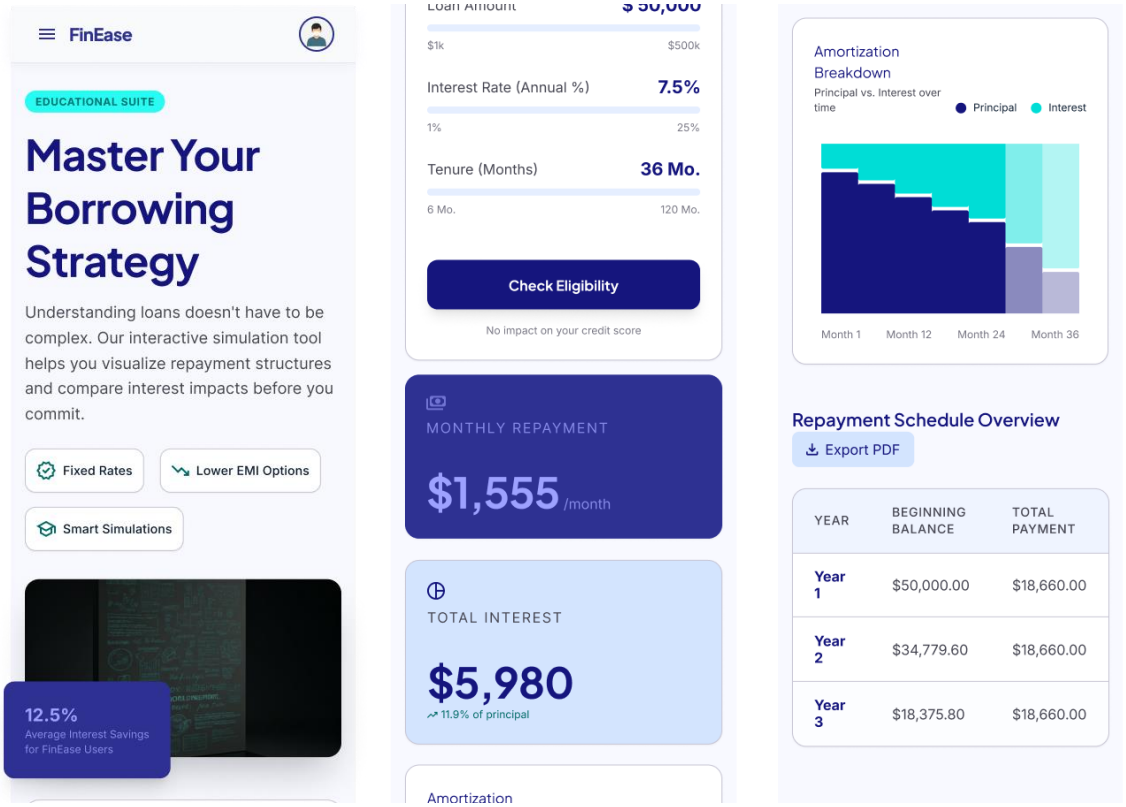

Figure 5.9 Loan Simulation Screen: Input Form and Results with Affordability Indicator

These screenshots display the input form for loan simulation feature. These also show the results section of the page with AI recommendations section and section for different statistics for better explanatory analysis.

5.6.8 Admin Panel Screen

The Admin Panel is accessible exclusively to admin and presents an administrative dashboard with distinct management sections. The NGO Programs tab displays all program listings (including unapproved ones invisible to regular users) with Add, Edit, and Delete actions. Submitting a status change atomically updates the Firestore document. The Analytics tab displays aggregated platform metrics including total users, active savings goals, lesson completion rates, and chatbot usage volumes, providing administrators with insight into platform engagement and content effectiveness.

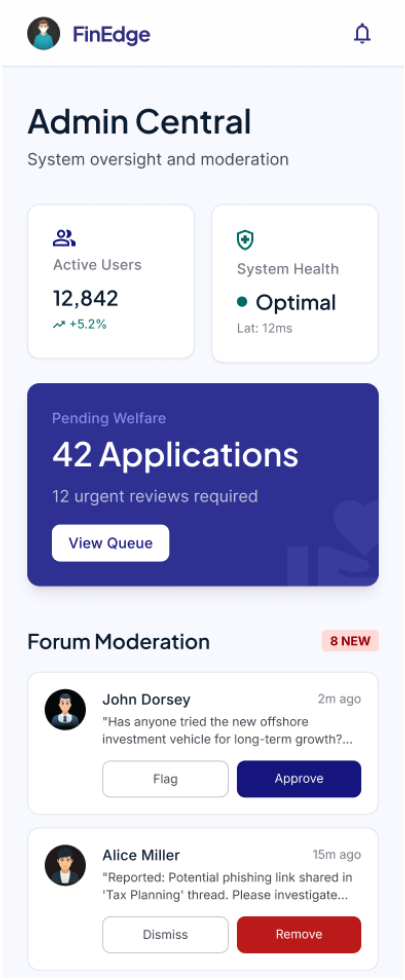

Figure 5.10 Admin Panel: Program Management, Application Review, and Analytics

This screenshot displays the admin-only interface showing NGO program management controls and the analytics dashboard with lesson completion rates and savings goal creation trends. The Admin Panel screens shown above present the NGO Program management list with add, edit, and delete controls and the analytics dashboard with key platform engagement metrics.

This diagram presents the feature first Flutter project directory structure adopted for FinEase, showing how each module's screens, widgets, and utilities are co located in their respective feature directories, with shared services and repositories in the core layer.

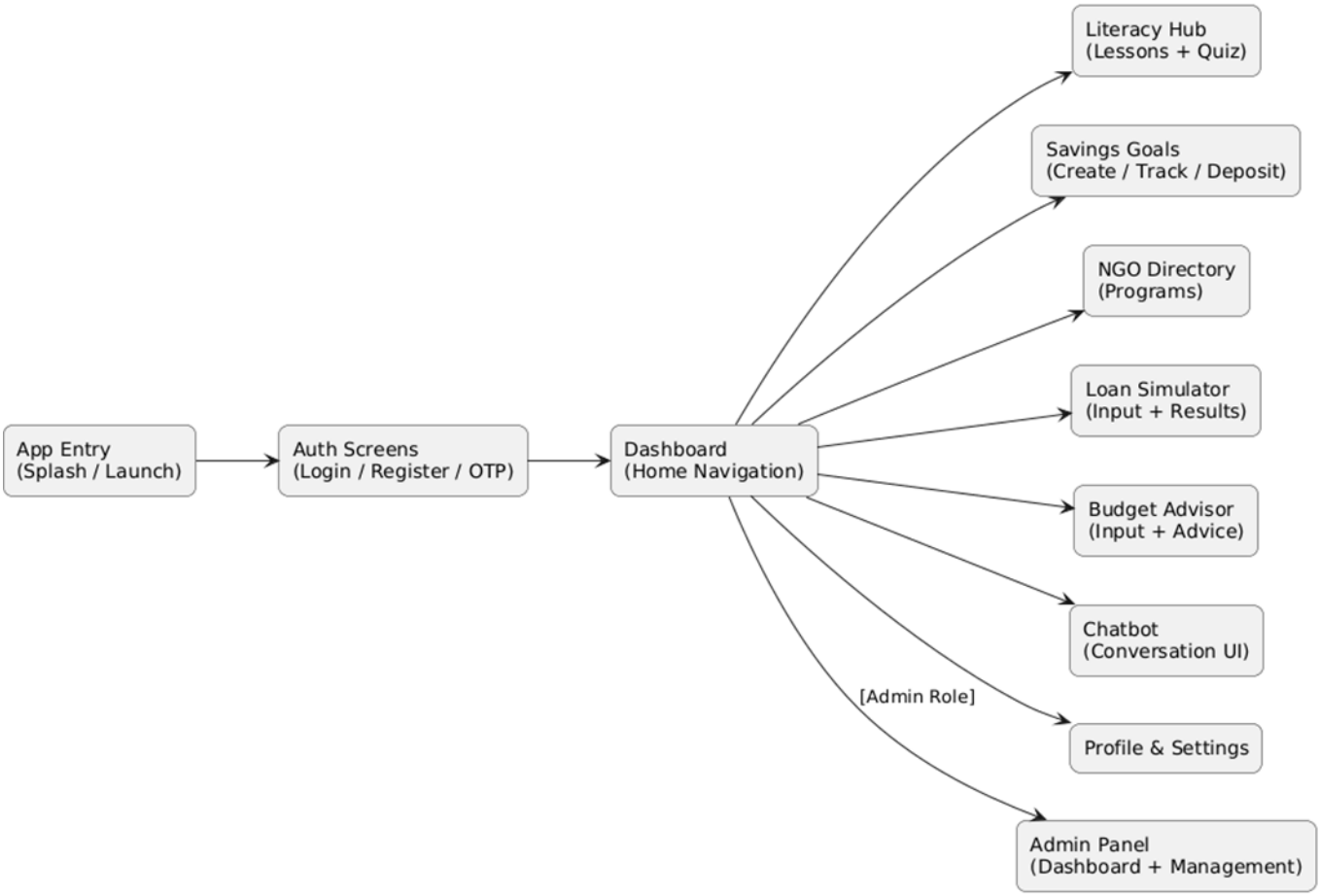

Figure 8.1: FinEase Application Screens: UI Overview

Figure 8.1 provides a consolidated visual overview of the key FinEase application screens, illustrating the consistent Material Design 3 visual language, Teal and Green color scheme, and bilingual interface across all major modules.

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

This chapter presents the complete testing and evaluation strategy for FinEase. Both manual and automated testing were conducted across all four project increments. Manual testing includes unit, functional, and integration testing. Automated testing uses Flutter's flutter_test framework and the Firebase Emulator Suite. The chapter concludes with the requirements traceability matrix linking all requirements to design components and test cases.

6.1 Testing Strategy

The development and testing processes are closely aligned across different phases to ensure quality and reliability. During the requirement analysis phase, acceptance testing is conducted, which involves manual validation of all functional and non-functional requirements against the running application. In the system design phase, system testing is performed to evaluate end-to-end functionality, ensuring that complete user journeys across all eleven modules work as intended. Moving to the module design phase, integration testing is carried out to verify interactions between different modules, as outlined in Table 6.10. During the implementation phase, unit testing focuses on validating individual service methods and algorithms, as detailed in Tables 6.1 through 6.4. Finally, in the coding phase, code review is conducted through peer evaluation, particularly emphasizing critical algorithms and security-sensitive components.

6.2 Unit Testing

6.2.1 Loan Simulation Tests

Table 6.1: Unit Test Results: Loan Simulation Calculation

No.	Test Case	Inputs	Expected	Actual	Status
UT 01	Standard 12 month loan 12% p.a.	P=100,000; r=12%; n=12	M=8,884.88; T=106,618; I=6,618	Correct	PASS
UT 02	Long term loan 18% p.a.	P=500,000; r=18%; n=60	M=12,687.53; T=761,252; I=261,252	Correct	PASS

No.	Test Case	Inputs	Expected	Actual	Status
UT 03	Interest free loan	P=60,000; r=0%; n=12	M=5,000; T=60,000; I=0	Correct	PASS
UT 04	Single month tenure	P=50,000; r=24%; n=1	M=51,000; T=51,000; I=1,000	Correct	PASS
UT 05	Zero principal	P=0; r=12%; n=12	M=0; T=0; I=0	Correct	PASS
UT 06	Large principal	P=5,000,000; r=8%; n=240	M=41,822; T=10,037,299; I=5,037,299	Correct	PASS

Table 6.1 presents test cases for the loan amortization algorithm, listing input parameters, expected and actual outputs for monthly installment and total repayable amount, and pass/fail status for all edge cases.

6.2.2 Quiz Scoring Tests

Table 6.2: Unit Test Results: Quiz Scoring and Points Award

No.	Test Case	Inputs	Expected	Actual	Status
UT 07	Perfect score, first quiz today	5/5 correct; threshold=80%; base=10pts; daily=0	100%; 10pts awarded	Correct	PASS
UT 08	Minimum pass score	4/5 correct; threshold=80%	80%; 10pts	Correct	PASS
UT 09	One below pass	3/5 correct; threshold=80%	60%; 0pts	Correct	PASS
UT 10	Daily cap full	Pass quiz; daily earned=50 (cap=50)	0pts (cap reached)	Correct	PASS

No.	Test Case	Inputs	Expected	Actual	Status
UT 11	Daily cap partial	Pass; daily=45; cap=50; base=10	5pts (limited)	Correct	PASS
UT 12	Daily reset	lastEarnedDate=yesterday; daily=50	Reset to 0; 10pts awarded	Correct	PASS

Table 6.2 covers unit tests for quiz score calculation and reward point award logic, including boundary conditions at the 79%/80% threshold and daily cap enforcement.

6.2.3 Savings Goal Progress Tests

Table 6.3: Unit Test Results: Savings Goal Progress Update

No.	Test Case	Inputs	Expected	Actual	Status
UT 13	First deposit on Created goal	Target=10,000; Current=0; Deposit=3,000	Current=3,000; Progress=30%; Active	Correct	PASS
UT 14	Mid progress deposit	Target=10,000; Current=6,000; Deposit=2,000	Current=8,000; Progress=80%; Active	Correct	PASS
UT 15	Goal completion	Target=10,000; Current=9,500; Deposit=600	Current=10,000; 100%; Completed	Correct	PASS
UT 16	Over deposit capped	Target=5,000; Current=4,800; Deposit=500	Current=5,000 (capped); Completed	Correct	PASS
UT 17	Zero deposit rejected	Any goal; Deposit=0	ValidationError raised	Error raised	PASS

No.	Test Case	Inputs	Expected	Actual	Status
UT 18	Negative deposit rejected	Any goal; Deposit= 500	ValidationError raised	Error raised	PASS

Table 6.3 verifies deposit recording logic including normal deposits, over deposit capping at the target amount, atomic transaction integrity, and goal completion status transition.

6.2.4 Budget Advice Generation Tests

Table 6.4: Unit Test Results: Budget Advice Generation

No.	Test Case	Inputs	Expected	Actual	Status
UT 19	Balanced budget	Income=50,000; Needs=22,000; Wants=13,000	1 positive advice item	1 item	PASS
UT 20	Over budget on needs	Income=50,000; Needs=30,000; Wants=12,000	Reduce needs by PKR 5,000	Correct	PASS
UT 21	Over budget on wants	Income=50,000; Needs=20,000; Wants=20,000	Reduce wants by PKR 5,000	Correct	PASS
UT 22	Under saving	Income=50,000; Needs=25,000; Wants=20,000	Save PKR 5,000 more per month	Correct	PASS
UT 23	Multiple issues cap at 5	All categories over budget	Maximum 5 items returned	5 items	PASS

Table 6.4 tests the 50/30/20 advice algorithm across balanced budgets, overspending scenarios, zero income edge cases, and confirms correct ranking order of generated advice items.

6.3 Functional Testing

6.3.1 User Authentication

Table 6.5: Functional Test Results: User Authentication

No.	Test Scenario	Input	Expected	Result
FT 01	Valid email registration	Email Adress	Account created; Dashboard loaded	PASS
FT 02	Incorrect Email	Invalid Email	Error shown; retry offered	PASS
FT 03	Duplicate email	Already registered email	Error: Account exists. Please login.	PASS
FT 04	Valid login	Registered email	Dashboard loaded with user data	PASS
FT 05	Blocked Email	Blocked Email	Error: Email is restricted. Request new one.	PASS
FT 06	Admin login	Admin role credentials	Admin Dashboard with controls	PASS
FT 07	Logout	Tap logout; force close; relaunch	Login screen; Dashboard inaccessible	PASS
FT 08	Language switch to Urdu	Change language preference	All UI switches to Urdu RTL	PASS

Table 6.5 Validates email OTP registration, duplicate rejection, OTP expiry handling, persistent session across application restarts, and role based dashboard routing

6.3.2 Savings Goal Module

Table 6.6: Functional Test Results: Savings Goal Module

No.	Test Scenario	Input	Expected	Result
FT 09	Create savings goal	Title, target=50,000; date=3 months	Goal at 0%; FCM scheduled	PASS
FT 10	Record first deposit	Select goal; deposit=10,000	Progress=20%; Active; real time update	PASS
FT 11	Complete goal	Deposit=40,000 on 10,000 current	Completed; reward issued; animation	PASS
FT 12	Cancel goal	Confirm cancel	Cancelled; in history	PASS
FT 13	Past date rejected	Target date = yesterday	Error: Target date must be in future	PASS
FT 14	Zero target rejected	Target = 0	Error: Target must be greater than zero	PASS
FT 15	Over deposit capped	Goal=5,000; current=4,800; deposit=500	Capped at 5,000; Completed	PASS

Table 6.6 verifies goal creation, deposit recording, over deposit capping, goal completion transition and cancellation flow.

6.3.3 NGO Directory

Table 6.7: Functional Test Results: NGO Directory

No.	Test Scenario	Input	Expected	Result
FT 16	Browse all programs	Open NGO Directory tab	All approved programs listed	PASS
FT 17	Filter by Financial Aid	Tap Financial Aid chip	Only Financial Aid programs shown	PASS
FT 18	Search by keyword	Type Ehsaas	Matching programs shown	PASS
FT 19	View program details	Tap any program card	Full eligibility, steps, contact shown	PASS
FT 20	Admin approves	Admin sets Approved; adds note	FCM sent to applicant; status updated	PASS

Table 6.7 covers program listing, category filtering, keyword search, application submission, draft saving, all status transitions, and admin review procedure end to end.

6.3.4 Loan Simulation

Table 6.8: Functional Test Results: Loan Simulation

No.	Test Scenario	Input	Expected	Result
FT 21	Standard loan	P=200,000; r=15%; n=24	Installment, total, interest in PKR	PASS
FT 22	Interest free loan	P=50,000; r=0%; n=10	M=5,000; T=50,000; I=0	PASS

No.	Test Scenario	Input	Expected	Result
FT 23	Affordability green	Income=40,000; M=10,000 (25%)	Green: Appears affordable	PASS
FT 24	Affordability amber	Income=40,000; M=16,000 (40%)	Amber: Moderately affordable	PASS
FT 25	Affordability red	Income=40,000; M=25,000 (63%)	Red: May strain your budget	PASS
FT 26	Save simulation	Run simulation; tap Save	Saved to Firestore; toast shown	PASS
FT 27	Non numeric input	Principal = abc	Inline validation error; submit blocked	PASS

Table 6.8 tests standard computation accuracy, zero interest edge case, all three affordability indicator color transitions, and optional result save to Firestore.

6.3.5 AI Chatbot

Table 6.9: Functional Test Results: AI Chatbot

No.	Test Scenario	Input	Expected	Result
FT 28	English query	How can I start saving money?	English advice within 8 s	PASS
FT 29	Urdu query	Urdu savings question	Urdu advice with RTL rendering	PASS
FT 30	NGO inquiry	Tell me about BISP eligibility	Accurate BISP information returned	PASS
FT 31	App navigation help	How do I use the loan calculator?	Step by step FinEase guide returned	PASS

No.	Test Scenario	Input	Expected	Result
FT 32	Offline graceful error	Message in airplane mode	Unable to connect. Check internet.	PASS
FT 33	Conversation context	4th message referencing message 1	Gemini shows awareness of context	PASS
FT 34	Typing indicator	Send any message	Spinner replaces send button while waiting	PASS

Table 6.9 validates English and Urdu query handling, multi turn context maintenance, typing indicator display and graceful error fallback on network failure.

6.4 Integration Testing

Table 6.10: Integration Test Results

No.	Integration Scenario	Modules Involved	Expected Behavior	Result
IT 01	Login to Personalized Dashboard	Auth, Profile, Dashboard	Dashboard shows username, goal summary, and featured lesson immediately	PASS
IT 02	Pass Quiz to Rewards Balance Update	LiteracyHub, RewardsService, Firestore	Points balance updates within 2 s of quiz pass	PASS
IT 03	Complete Goal to Reward Issued	SavingsService, RewardsService, Firestore transaction	Completion AND reward updated atomically; no inconsistency	PASS
IT 04	Budget Input to BudgetPlan Stored	BudgetAdvisorService, BudgetRepository, Firestore	Advice generated AND BudgetPlan verifiably created in Firestore	PASS

No.	Integration Scenario	Modules Involved	Expected Behavior	Result
IT 05	Chatbot to Gemini to Stored	ChatbotService, Cloud Functions, Gemini, Firestore	Response rendered; ChatbotSession Firestore document updated	PASS
IT 06	Goal Creation to FCM Scheduled	SavingsService, Cloud Functions, FCM	Cloud Function onCreate trigger schedules FCM for target date 1 day	PASS
IT 07	Offline Lesson Access	LiteracyHub, Firestore cache	Previously loaded lessons accessible in airplane mode	PASS
IT 08	Language Switch to Global UI Update	Profile, Provider, all screens	All app text switches to Urdu RTL after language update	PASS
IT 09	Daily Points Cap Reset	RewardsService, QuizService, Firestore	After midnight, daily counter resets before awarding points	PASS

Table 6.10 presents eleven cross-module integration scenarios covering quiz-to-rewards flow, goal completion with reward, chatbot session persistence and offline lesson access.

6.5 Automated Testing

Table 6.11: Automated Testing Tools and Results

Tool	Type	Scope	Tests	Pass Rate
flutter_test (unit)	Unit Testing	LoanSimulationService (6), QuizService (6), SavingsService (6), BudgetAdvisorService (5) all algorithms	23	100%
flutter_test (widget)	Widget Testing	LoginScreen form validation, SavingsGoalCard progress bar,	8	100%

Tool	Type	Scope	Tests	Pass Rate
		QuizScreen answer selection, BudgetAdvisorStep1 validation		
Firebase Emulator Suite	Integration Testing	Firebase Auth OTP flows, Firestore CRUD with Security Rules, Cloud Function invocations with mock Gemini responses	12	100%
Firebase Security Rules Tests	Security Testing	All 14 collection Security Rules: user scoped isolation, admin only paths, unauthenticated rejection	18	100%

Table 6.11 summarizes the complete automated test suite across units, widget, integration, and Security Rules categories, showing the tool used, number of test cases, and 100% pass rate across all 61 automated tests. It also summarizes the complete automated test suite across unit, widget, integration, and Security Rules categories, showing tool used, number of test cases, and pass rate 61 tests total at 100%. Total automated test coverage: 61 automated test cases (23 unit + 8 widget + 12 integration + 18 security rules) all passing at 100%. Test files are organized under test/unit/services/, test/unit/widgets/, test/integration/, and test/security/ in the GitHub repository.

6.6 Requirements Traceability Matrix

Table 7.1: Software Requirements Traceability Matrix

Req. ID	Requirement Summary	Design Component	Test Cases
FR UA 01	Email registration	AuthService, UserRepository, Figure 4.8	FT 01, FT 02, FT 03, Table 6.5
FR UA 03	Login with Email	AuthService, Figure 4.8	FT 04, FT 05, IT 01

Req. ID	Requirement Summary	Design Component	Test Cases
FR UA 05	Role-based access control	AuthService, Firebase Custom Claims	FT 06, IT 02 (admin path)
FR UA 06	Profile language update	ProfileRepository, Provider propagation	FT 08, IT 10
FR FL 01	Categorised lessons	LiteracyService, LessonRepository, Figure 4.2	Table 6.2, lesson browser test
FR FL 05	Award points for quiz pass	QuizService, RewardsService, Firestore transaction	UT 07 to UT 12, IT 02
FR FL 07	Offline lesson access	Firestore offline persistence	IT 09
FR SG 01	Create savings goals	SavingsService, SavingsRepository, Figure 4.3	FT 09, FT 13, FT 14, Table 6.6
FR SG 02	Atomic deposit recording	SavingsService.runTransaction(), Section 5.4.2	UT 13 to UT 18, FT 10
FR SG 04	Auto complete on target	SavingsService, Section 5.4.2	UT 15, UT 16, FT 11, IT 03
FR SG 05	FCM savings reminder	Cloud Function onCreate, FCM	IT 07
FR NG 01	NGO directory	NGOService, NGORepository, Figure 4.13	FT 16 to FT 19, Table 6.7
FR NG 02	Admin listing management	AdminService, Cloud Function onUpdate	FT 20
FR LS 02	Amortization formula	LoanSimulationService, Section 5.4.1	UT 01 to UT 06, FT 23 to FT 27
FR LS 04	Zero interest edge case	LoanSimulationService zero branch	UT 03, FT 24

Req. ID	Requirement Summary	Design Component	Test Cases
FR LS 05	Affordability indicator	LoanSimulationService + ProfileRepository	FT 25, FT 26, FT 27
FR BA 02	50/30/20 budget rule	BudgetAdvisorService, Section 5.4.3	UT 19 to UT 23, IT 05
FR BA 04	Max 5 advice items	BudgetAdvisorService (cap)	UT 23
FR RW 01	Award points on quiz/goal	RewardsService, Firestore transaction	UT 07, FT 11, IT 02, IT 03
FR RW 02	Daily points cap	RewardsService, Section 5.4.4	UT 10, UT 11
FR RW 05	Daily cap reset	RewardsService (date comparison)	UT 12, IT 11
FR CH 01	Gemini chatbot	ChatbotService, Cloud Function, Figure 4.12	FT 30, FT 32, FT 33, IT 06
FR CH 02	Bilingual responses	Cloud Function language param, Gemini	FT 30, FT 31
FR CH 05	Conversation history	ChatbotRepository, Firestore	FT 35, IT 06
FR CH 06	Graceful offline error	ChatbotService error handling	FT 34
NFR SE 02	Per user data isolation	Firestore Security Rules	Security Rules Tests (18/18)
NFR SE 03	API key not in client	Cloud Function secrets, code review	Code review and APK inspection verified
NFR RL 01	Offline lesson access	Firestore offline persistence	IT 09
NFR US 03	Bilingual Urdu/English UI	Provider language switching, RTL	FT 08, IT 10

Table 7.1 maps every functional and non functional requirement to its corresponding design component, implementation location, and test case(s), ensuring complete bidirectional traceability across the project

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

FinEase has been designed, developed, and tested as a comprehensive mobile first financial inclusion platform for underserved communities in Pakistan. The project sets out to address a multi dimensional problem low financial literacy, limited awareness of welfare programs, lack of personal financial management tools, and absence of accessible AI powered financial guidance through a single, integrated, bilingual Android application. This goal was fully achieved across all four development stages.

The technology choices Flutter, Firebase, and Gemini worked well for this project. Flutter allowed fast and clean UI development with full bilingual support and RTL text direction from a single codebase. Firebase's serverless architecture removed server management needs while providing live updates and offline access. The Gemini API's strong Urdu language performance made it the best choice for bilingual chat.

The Agile inspired incremental model worked well, letting the team deliver working software at each milestone while getting regular feedback. Each increment produced a testable vertical slice of functionality validated before the next beginning. Beyond technical achievements, FinEase represents a meaningful response to a genuine social challenge giving underserved users access to financial lessons, savings tools, welfare information, loan help, and AI guidance through a free, bilingual, offline capable application.

7.2 Future Work

7.2.1 Real Financial Transaction Integration

The most useful future improvement would be integrating FinEase with JazzCash or Easypaisa APIs to enable actual financial transactions savings deposits to a linked mobile wallet, bill payment tracking, and welfare disbursement confirmation. Integration with Pakistan's Raast instant payment system would additionally enable free real time bank transfers, transforming FinEase from an advisory platform into a full financial services application.

7.2.2 Regional Language Expansion

Extending the Urdu/English interface to support Punjabi, Sindhi, Pashto, and Balochi would significantly expand FinEase's reach to approximately 210 million Pakistanis who speak one of the five major languages. Google Fonts provides Noto typefaces for all these scripts, and the Gemini API already supports these languages where the main work would be translating the UI and lessons, not changing the AI.

7.2.3 Machine Learning Based Personalisation

Future plans include personalised recommendations based on user behavior lesson completion patterns, savings goal performance, chatbot query history to proactively suggest relevant content, savings strategies, and NGO programs. Firebase's integration with Google's Vertex AI provides a natural path for implementing on device or cloud based personalisation without a separate ML infrastructure.

7.2.4 iOS Platform Release and Advanced AI Counselling

The Flutter codebase is already structured for iOS compilation. A future increment would address iOS specific configuration, package and sign the iOS build, and submit to the App Store, extending FinEase's reach to iPhone users. A real time video consultation module could additionally connect FinEase users with certified financial counsellors for complex decisions extending the platform beyond automated AI advice to include professional human oversight where stakes require it.

7.2.5 Offline First Architecture Enhancement

A more comprehensive offline first enhancement would enable full functionality without connectivity: pre downloading all lesson content, quiz data, and a curated set of NGO program listings on first launch; storing loan simulation results and budget advice entirely locally; and queuing savings goal deposits and chatbot queries for batch submission when connectivity is restored. This is particularly important for users in rural areas with intermittent or no mobile data coverage.

7.2.6 Investment Education and Advanced Analytics

A future Investment Education module would cover Pakistan's National Savings Schemes (Special Savings Certificates, Defence Savings Certificates), basic investment concepts, and a Micro Investment Simulator demonstrating compound interest growth over time.

References

Demirguc Kunt, A., Klapper, L., Singer, D., Ansar, S. & Hess, J. (2018). The Global Findex Database 2017: Measuring Financial Inclusion and the Fintech Revolution. World Bank Group.

Beck, T., & Demirguc Kunt, A. (2008). Access to Finance: An Unfinished Agenda. Review of Finance, 12(3), 383–396.

Khwaja, A. I., & Mian, A. (2008). Tracing the Impact of Bank Liquidity Shocks. American Economic Review, 98(4), 1413–1442.

Mbiti, I., & Weil, D. N. (2015). Mobile Banking: The Impact of M Pesa in Kenya. American Economic Review, 101(3), 253–259.

Hussain, J., Salia, S., & Karim, A. (2019). Is Knowledge That Powerful? Financial Literacy and Access to Finance. Journal of Small Business and Enterprise Development.

Lusardi, A., & Mitchell, O. S. (2014). The Economic Importance of Financial Literacy. Journal of Economic Literature, 52(1), 5–44.

Kaiser, T., & Menkhoff, L. (2017). Does Financial Education Impact Financial Literacy and Financial Behavior? The Economic Journal, 127(601), 1905–1934.

Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From Game Design Elements to Gamefulness: Defining Gamification. In Proc. MindTrek 2011 (pp. 9–15). ACM.

Fernandes, D., Lynch, J. G., & Netemeyer, R. G. (2014). Financial Literacy, Financial Education, and Downstream Financial Behaviors. Management Science, 60(8), 1861–1883.

Philippon, T. (2019). On Fintech and Financial Inclusion. NBER Working Paper No. 26330.

Gnewuch, U., Morana, S., & Maedche, A. (2017). Towards Designing Cooperative and Social Conversational Agents for Customer Service. In Proc. ICIS 2017. AIS.

Engelberg, J., & Parsons, C. A. (2016). Worrying about the Power of Words. Review of Financial Studies, 29(7), 1687–1730.

Warren, E., & Tyagi, A. W. (2005). All Your Worth: The Ultimate Lifetime Money Plan. Free Press.

Muralidharan, K., Niehaus, P., & Sukhtankar, S. (2016). Building State Capacity: Evidence from Biometric Smartcards in India. American Economic Review, 106(10), 2895–2929.

Banerjee, A., & Duflo, E. (2011). Poor Economics: A Radical Rethinking of the Way to Fight Global Poverty. PublicAffairs.

OECD/INFE. (2021). OECD/INFE 2021 International Survey of Adult Financial Literacy. OECD.

World Bank. (2021). The Global Findex Database 2021 : Financial Inclusion, Digital Payments, and Resilience. World Bank Group.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

Boehm, B., & Turner, R. (2004). Balancing Agility and Discipline. Addison Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object Oriented Software. Addison Wesley.

Garrett, J. J. (2011). The Elements of User Experience (2nd ed.). New Riders.

Myers, G. J., Sandler, C., & Badgett, T. (2012). The Art of Software Testing (3rd ed.). Wiley.

Flutter Team. (2024). Flutter Documentation. Google LLC. https://flutter.dev/docs

Firebase. (2024). Firebase Documentation. Google LLC. https://firebase.google.com/docs

Google. (2024). Gemini API Documentation. https://ai.google.dev/docs

State Bank of Pakistan. (2023). Pakistan Financial Inclusion Survey 2022 . SBP.